

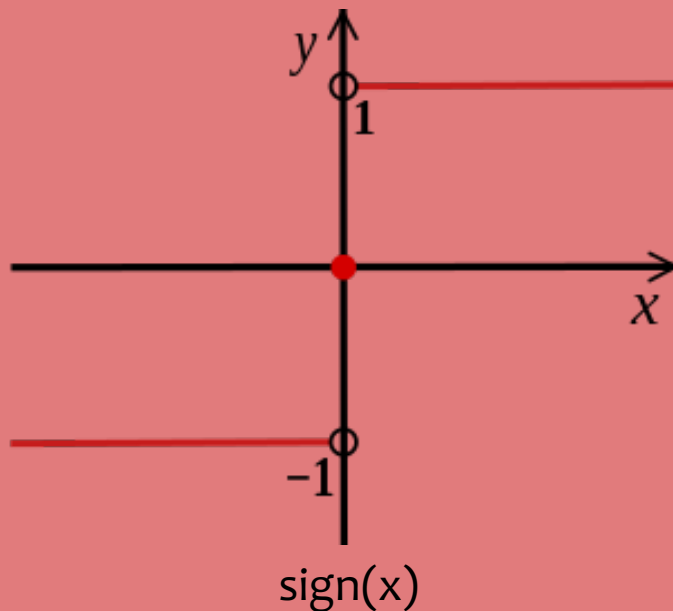
Introduction to Neural Network

LOGISTIC REGRESSION

Predicting a Binary Response Y Using X

This decision (activation) function isn't differentiable:

$$h(\mathbf{x}) = \text{sign}(\boldsymbol{\theta}^T \mathbf{x})$$



Slide from Matt Gormley, CMU

Use a differentiable function instead:

$$p_{\boldsymbol{\theta}}(y = 1 | \mathbf{x}) = \frac{1}{1 + \exp(-\boldsymbol{\theta}^T \mathbf{x})}$$

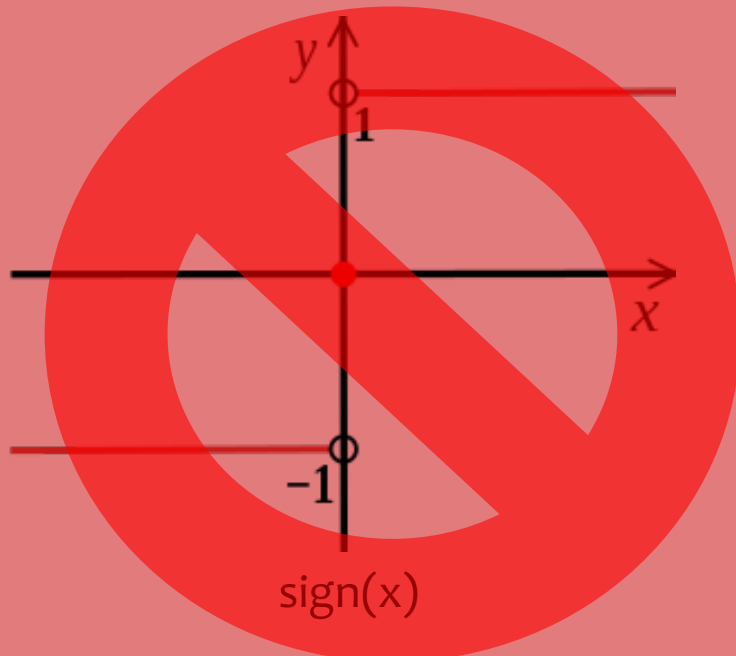


$$\text{logistic}(u) \equiv \frac{1}{1 + e^{-u}}$$

Predicting a Binary Response Y Using X

This decision (activation) function isn't differentiable:

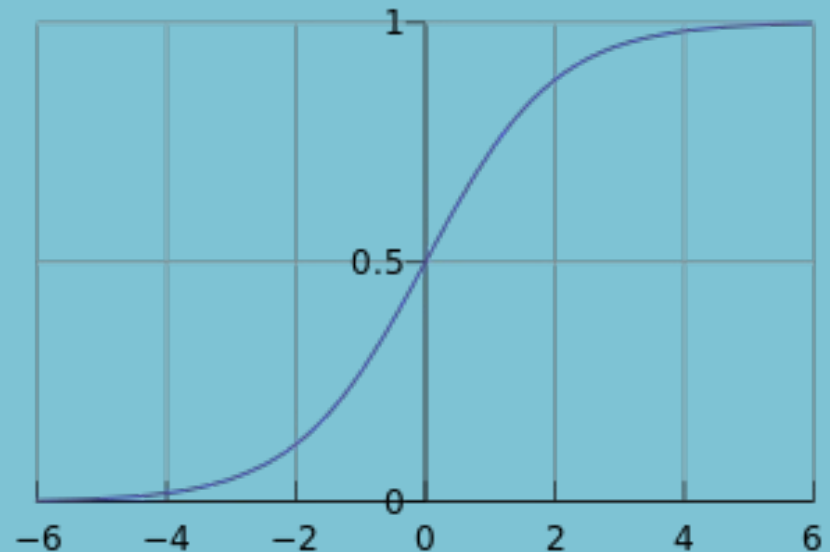
$$h(\mathbf{x}) = \text{sign}(\boldsymbol{\theta}^T \mathbf{x})$$



Slide from Matt Gormley, CMU

Use a differentiable function instead:

$$p_{\boldsymbol{\theta}}(y = 1|\mathbf{x}) = \frac{1}{1 + \exp(-\boldsymbol{\theta}^T \mathbf{x})}$$



$$\text{logistic}(u) \equiv \frac{1}{1 + e^{-u}}$$

Logistic Regression

Data: Inputs are continuous vectors of length K . Outputs are discrete.

$$\mathcal{D} = \{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^N \text{ where } \mathbf{x} \in \mathbb{R}^K \text{ and } y \in \{0, 1\}$$

Model: Logistic function applied to dot product of parameters with input vector.

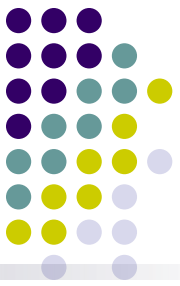
$$p_{\boldsymbol{\theta}}(y = 1|\mathbf{x}) = \frac{1}{1 + \exp(-\boldsymbol{\theta}^T \mathbf{x})}$$

Learning: finds the parameters that minimize some objective function. $\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} J(\boldsymbol{\theta})$

Prediction: Output is the most probable class.

$$\hat{y} = \underset{y \in \{0,1\}}{\operatorname{argmax}} p_{\boldsymbol{\theta}}(y|\mathbf{x})$$

NEURAL NETWORKS

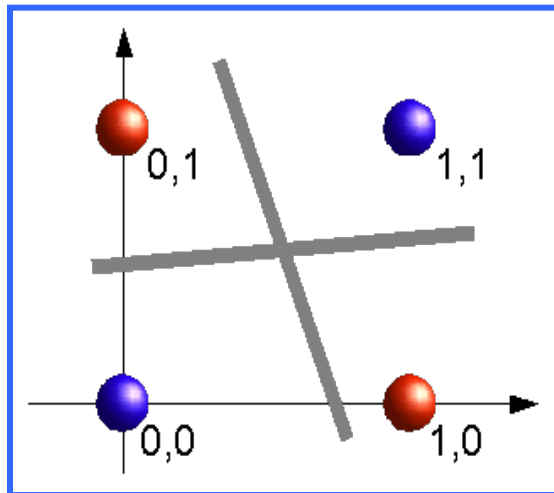
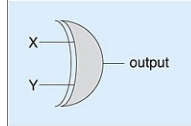


Learning highly non-linear functions

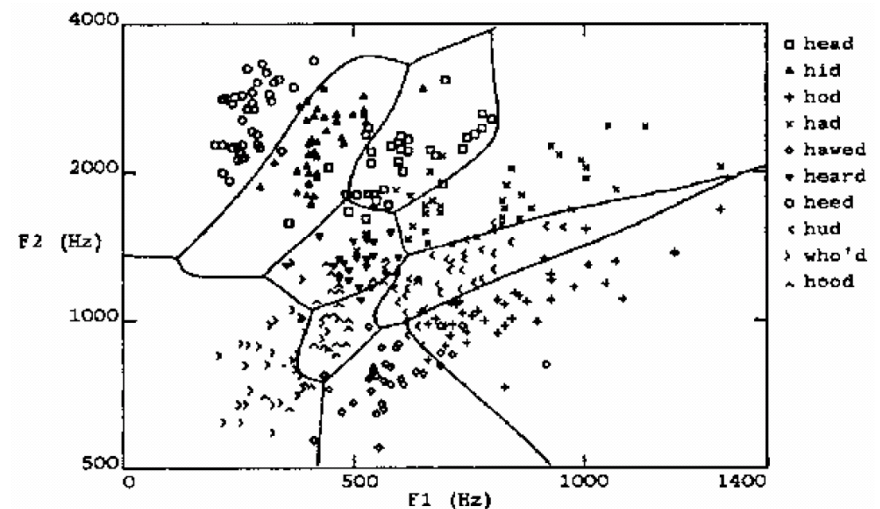
$f: X \rightarrow Y$

- f might be non-linear function
- X (vector of) continuous and/or discrete vars
- Y (vector of) continuous and/or discrete vars

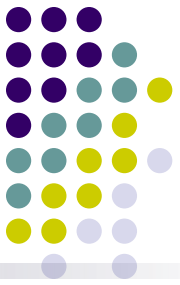
The XOR gate



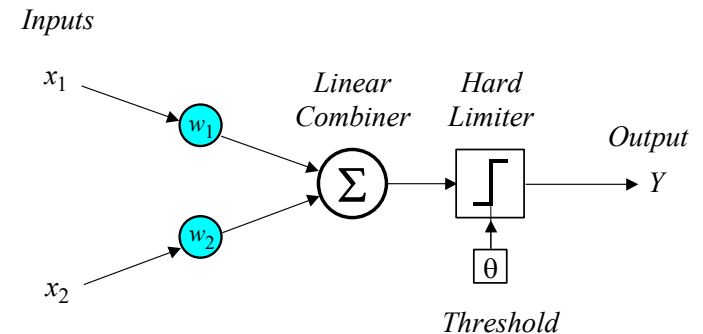
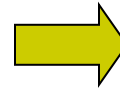
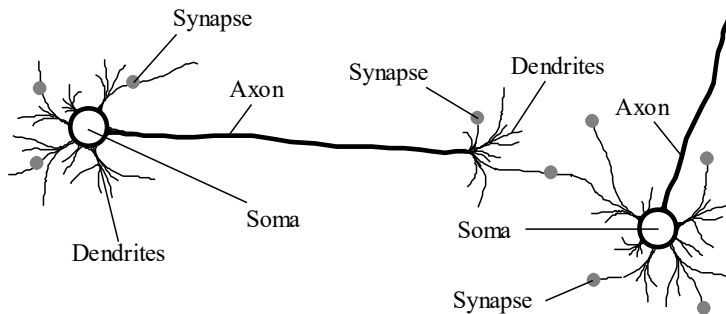
Speech recognition



Perceptron and Neural Nets



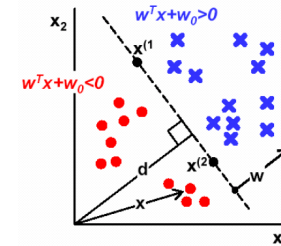
- From biological neuron to artificial neuron (perceptron)



- Activation function

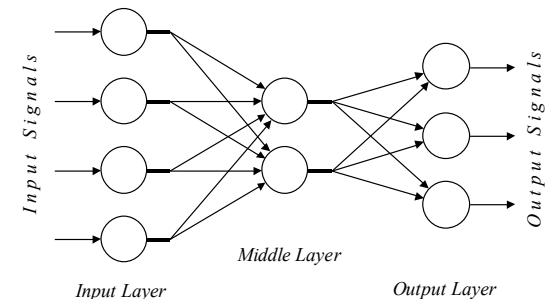
$$X = \sum_{i=1}^n x_i w_i$$

$$y = \begin{cases} +1, & \text{if } X + w_0 > 0 \\ -1, & \text{if } X + w_0 < 0 \end{cases}$$



- Artificial neuron networks

- supervised learning
- gradient descent

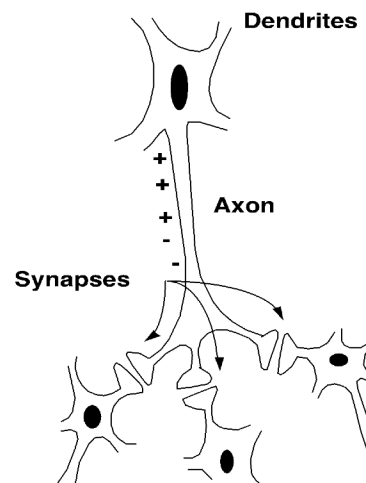


Connectionist Models

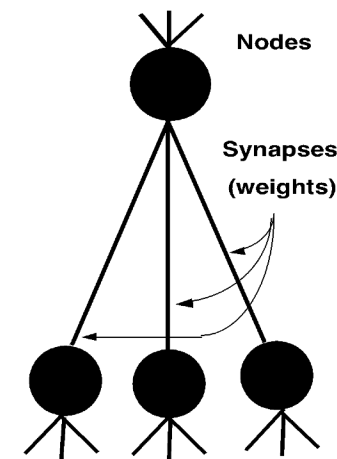


- Consider humans:

- Neuron switching time
~ 0.001 second
- Number of neurons
~ 10^{10}
- Connections per neuron
~ 10^{4-5}
- Scene recognition time
~ 0.1 second
- 100 inference steps doesn't seem like enough
→ much parallel computation



↓
model



- Properties of artificial neural nets (ANN)

- Many neuron-like threshold switching units
- Many weighted interconnections among units
- Highly parallel, distributed processes

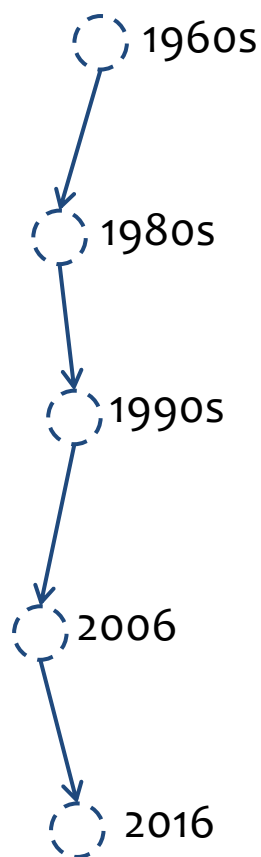
Motivation

Why is everyone talking about Deep Learning?

- “Deep learning is a class of machine learning algorithms that uses multiple layers to progressively extract higher-level features from the raw input.”
- An example of deep learning is a deep neural network with multiple layers between the input and output layers.



Why is everyone talking about Deep Learning?



Deep learning:

- Has won numerous pattern recognition competitions
- Does so with minimal feature engineering

This wasn't always the case!

Since 1980s: Form of models hasn't changed much, but lots of new tricks...

- More hidden units
- Better (online) optimization
- New nonlinear functions (ReLUs)
- Faster computers (CPUs and GPUs)

Background

A Recipe for Machine Learning

1. Given training data:

$$\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^N$$

2. Choose each of these:

- Decision (activation) function

$$\hat{\mathbf{y}} = f_{\theta}(\mathbf{x}_i)$$

- Loss function

$$\ell(\hat{\mathbf{y}}, \mathbf{y}_i) \in \mathbb{R}$$

Slide from Matt Gormley, CMU

Face



Face



Not a face



Examples: Linear regression,
Logistic regression, Neural Network

Examples: Mean-squared error,
Cross Entropy

Background

A Recipe for Machine Learning

1. Given training data:

$$\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^N$$

2. Choose each of these:

- Decision (activation) function

$$\hat{\mathbf{y}} = f_{\boldsymbol{\theta}}(\mathbf{x}_i)$$

- Loss function

$$\ell(\hat{\mathbf{y}}, \mathbf{y}_i) \in \mathbb{R}$$

3. Define goal:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \sum_{i=1}^N \ell(f_{\boldsymbol{\theta}}(\mathbf{x}_i), \mathbf{y}_i)$$

4. Train with Stochastic gradient descent (SGD):
(take small steps opposite the gradient)

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta_t \nabla \ell(f_{\boldsymbol{\theta}}(\mathbf{x}_i), \mathbf{y}_i)$$

Background

A Recipe for Gradients

1. Given training data

$$\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^N$$

2. Choose each of the

- Decision function

$$\hat{\mathbf{y}} = f_{\boldsymbol{\theta}}(\mathbf{x}_i)$$

- Loss function

$$\ell(\hat{\mathbf{y}}, \mathbf{y}_i) \in \mathbb{R}$$

Backpropagation can compute this gradient!

And it's a **special case of a more general algorithm** called reverse-mode automatic differentiation that can compute the gradient of any differentiable function efficiently!

opposite the gradient)


$$\boldsymbol{\theta}^{(t)} \leftarrow \boldsymbol{\theta}^{(t)} - \eta_t \nabla \ell(f_{\boldsymbol{\theta}}(\mathbf{x}_i), \mathbf{y}_i)$$

Goals for Today's Lecture

1. Study the types of **activation functions (decision functions)** – with special attention to the **logistic function**
2. Study the types of loss functions – with special attentions to SSE and CE (cross-entropy = negative log likelihood)
3. Find the link between Perceptron and Logistic regression

– Decision function

$$\hat{y} = f_{\theta}(x_i)$$

– Loss function

$$\ell(\hat{y}, y_i) \in \mathbb{R}$$

4. Train with SGD:

– Take small steps
opposite the gradient)

$$\theta^{(t+1)} = \theta^{(t)} - \eta_t \nabla \ell(f_{\theta}(x_i), y_i)$$

Linear Regression

Slide modified from Matt Gormley, CMU

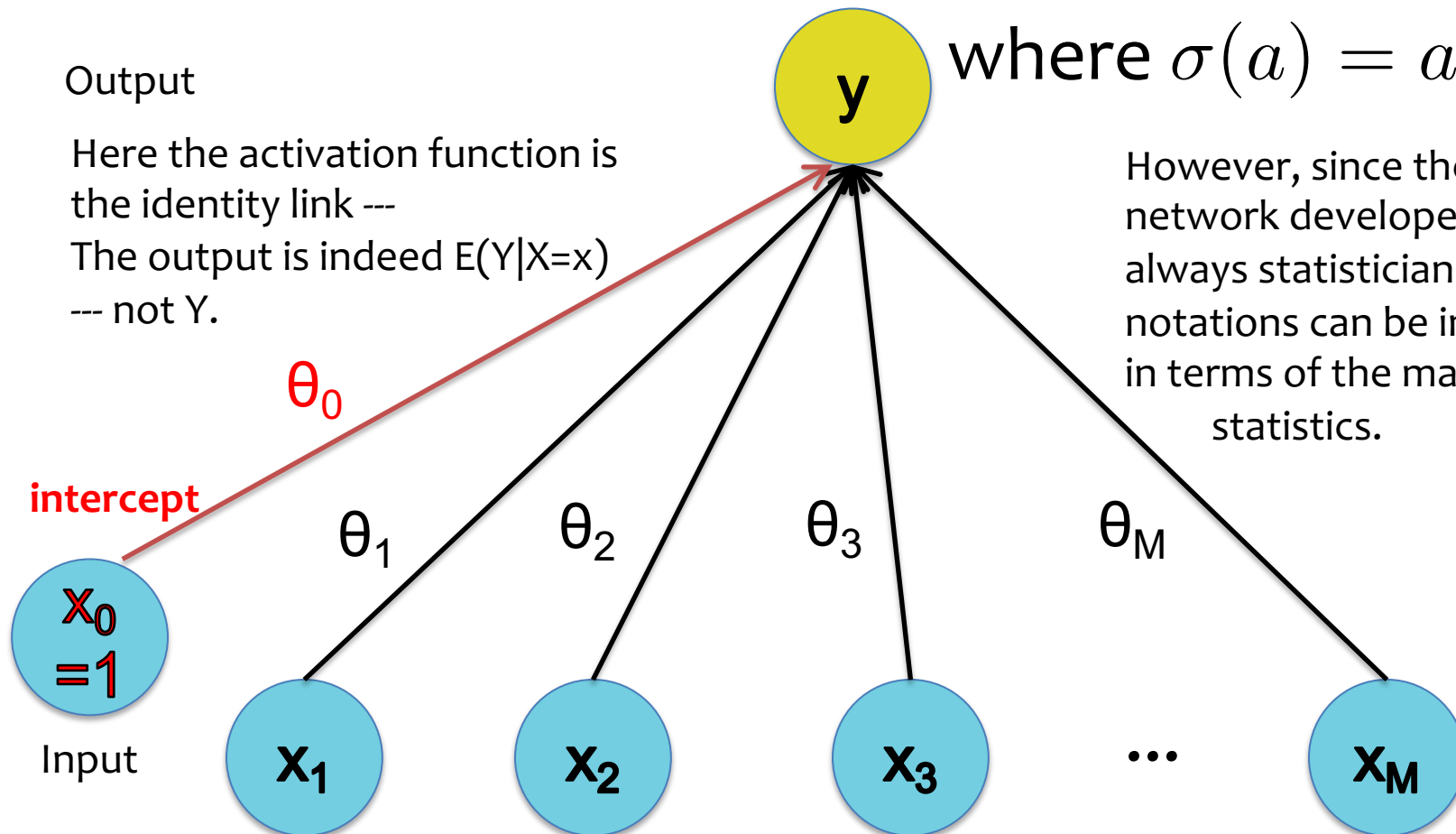
$$y = h_{\theta}(x) = \sigma(\theta^T x)$$

Output

Here the activation function is
the identity link ---
The output is indeed $E(Y|X=x)$
--- not Y .

where $\sigma(a) = a$

However, since the neural
network developers are not
always statisticians, the
notations can be inaccurate
in terms of the mathematics/
statistics.



Decision/Activation Functions

Logistic Regression

Slide modified from Matt Gormley, CMU

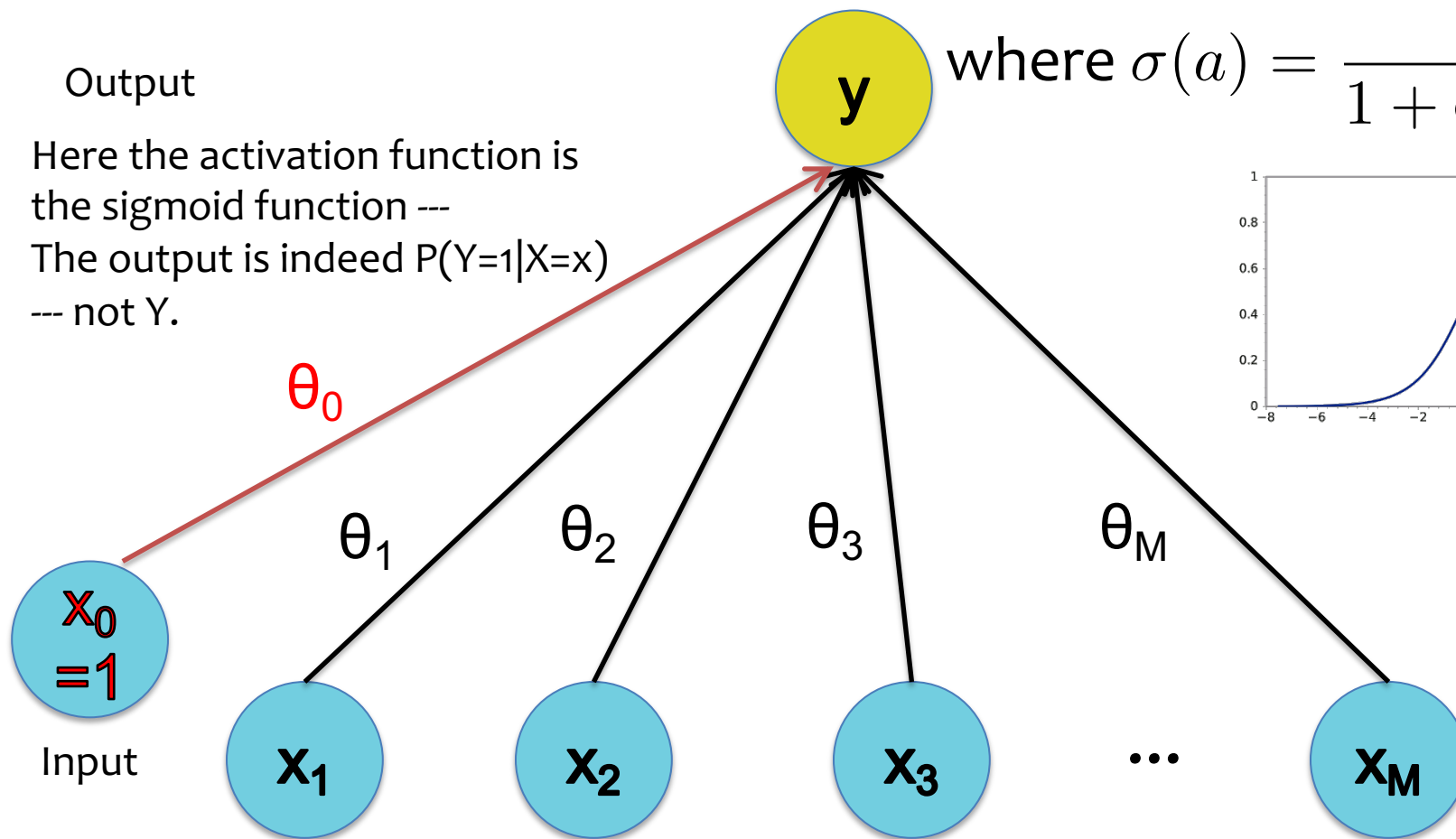
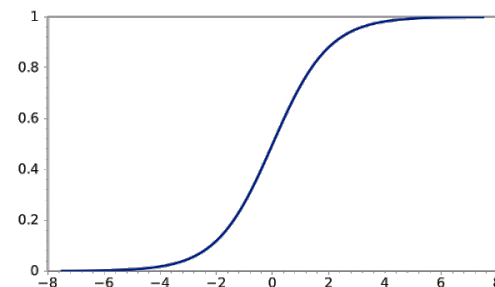
$$y = h_{\theta}(\mathbf{x}) = \sigma(\theta^T \mathbf{x})$$

$$\text{where } \sigma(a) = \frac{1}{1 + \exp(-a)}$$

Output

Here the activation function is the sigmoid function ---

The output is indeed $P(Y=1|X=x)$ --- not Y .

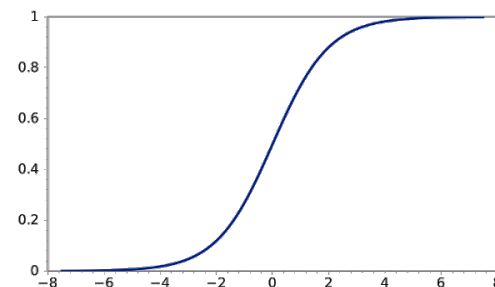


Logistic Regression

Output

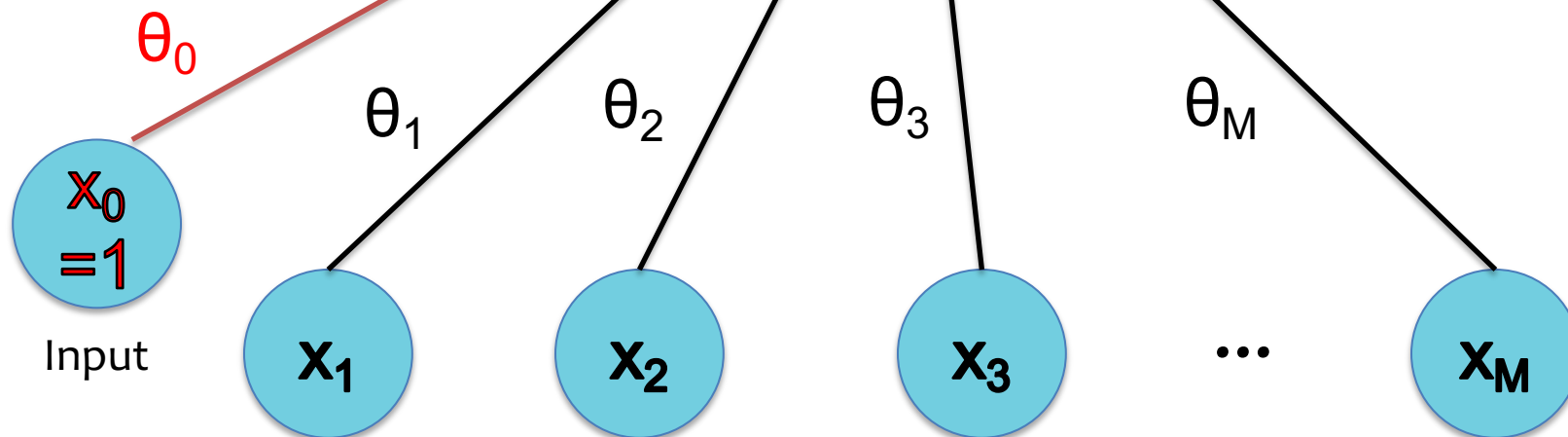
$$y = h_{\theta}(x) = \sigma(\theta^T x)$$

$$\text{where } \sigma(a) = \frac{1}{1 + \exp(-a)}$$



Logistic Regression = Perceptron:

- (1) without hidden layer
- (2) With loss function = negative log likelihood
(also called the cross-entropy, or CE)
- (3) with activation function = logistic

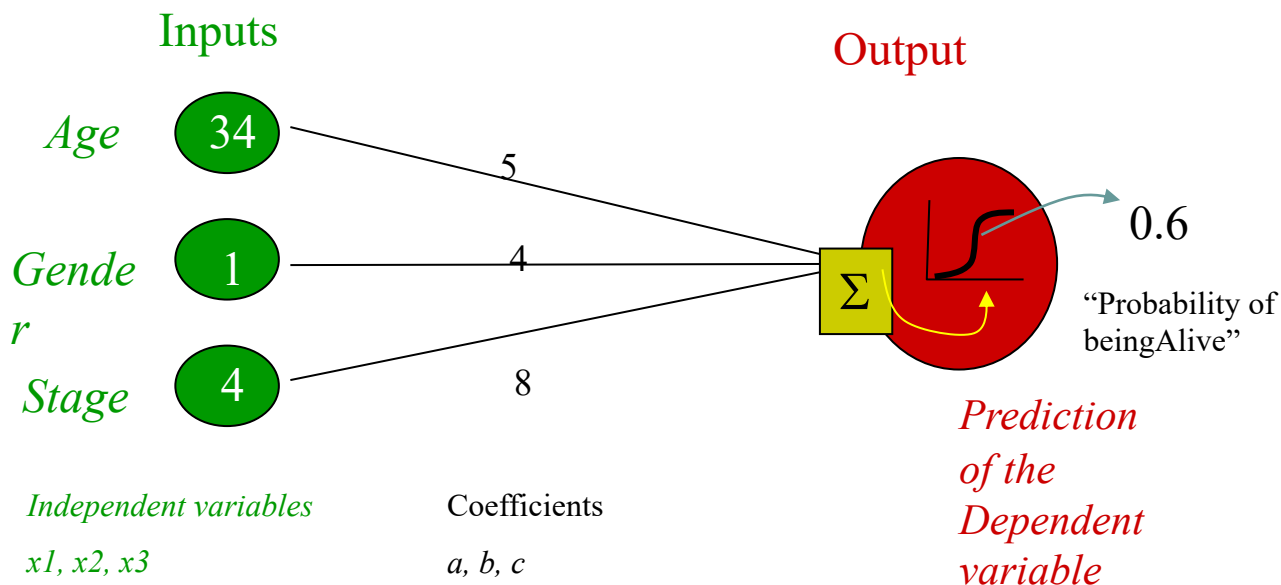


Neural Network Model (with no hidden layer) = Logistic Regression

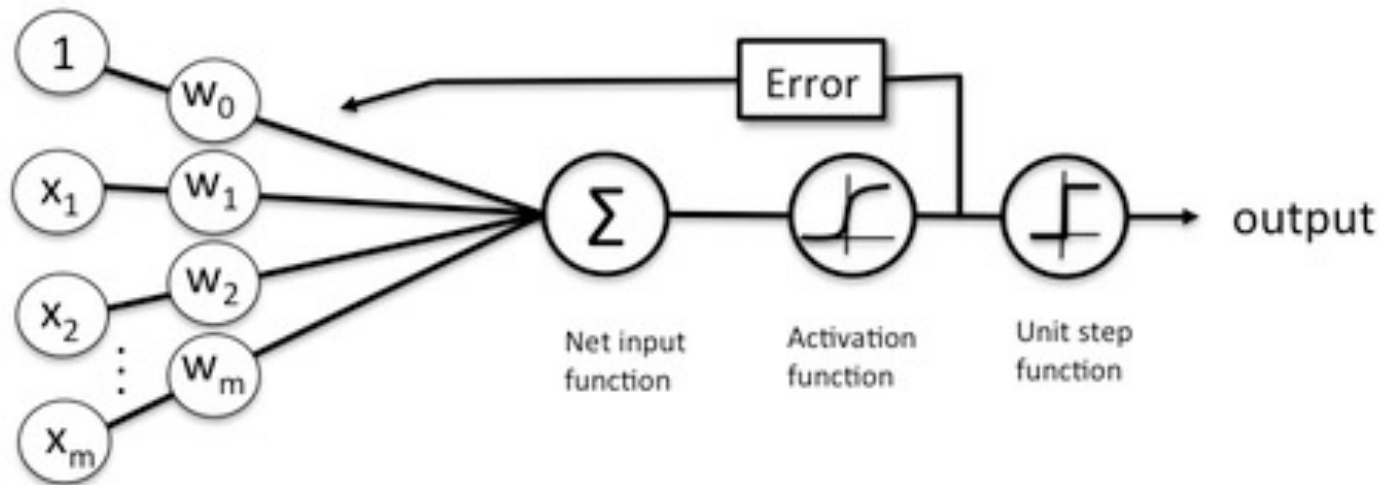
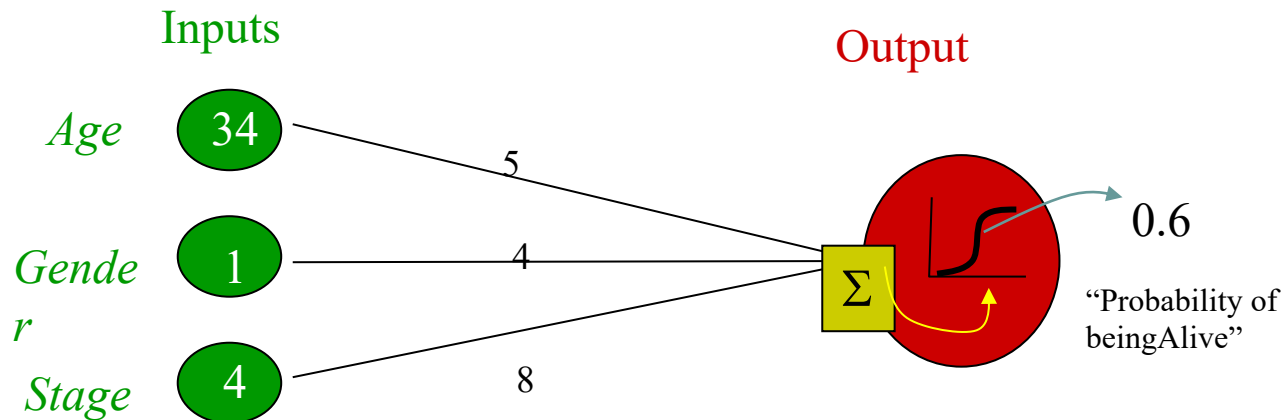
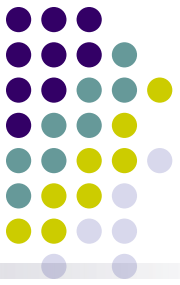


- Independent variable = input variable
- Dependent variable $\neq$ output
- Coefficients = “weights” (* the intercept θ_0 is also called the “bias” *)
- Loss function = Cross-entropy (CE) (i.e. negative log likelihood)

Logistic Regression Model (the sigmoid unit)

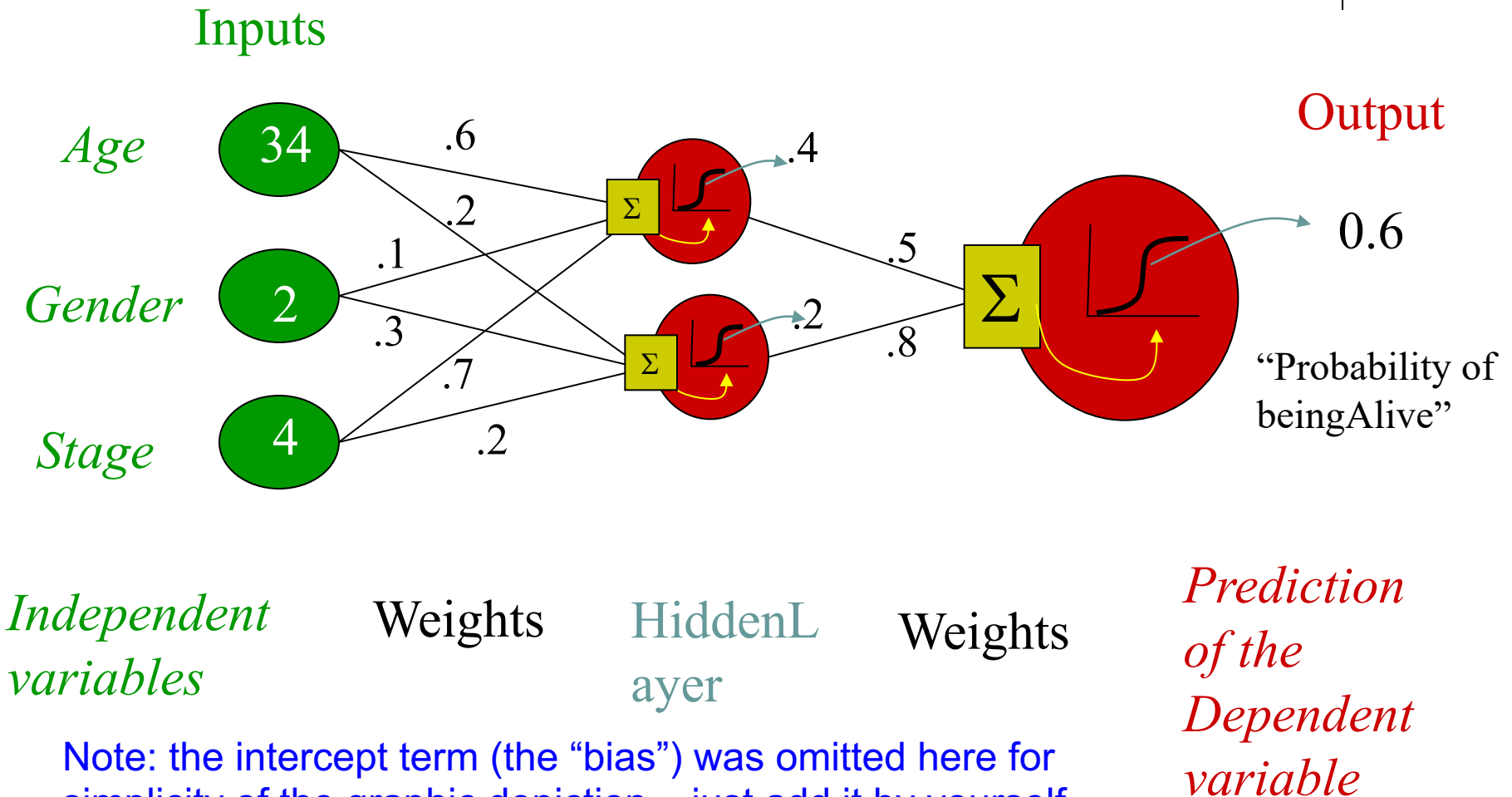
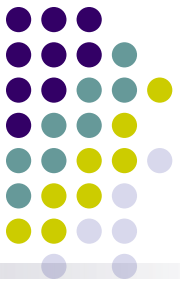


Neural Network Model (with no hidden layer) = Logistic Regression



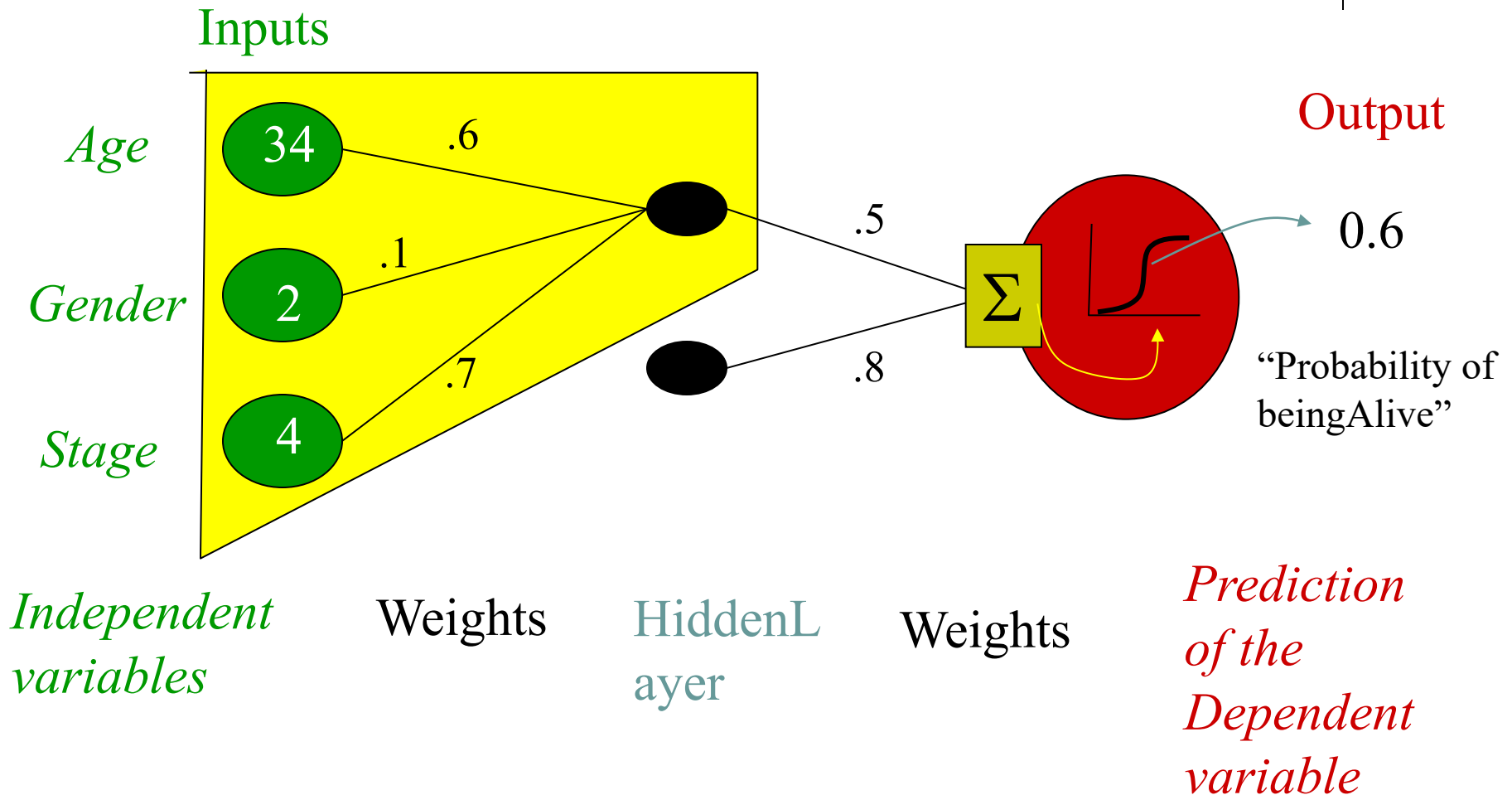
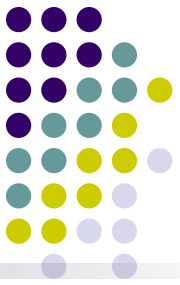
Schematic of a logistic regression classifier.

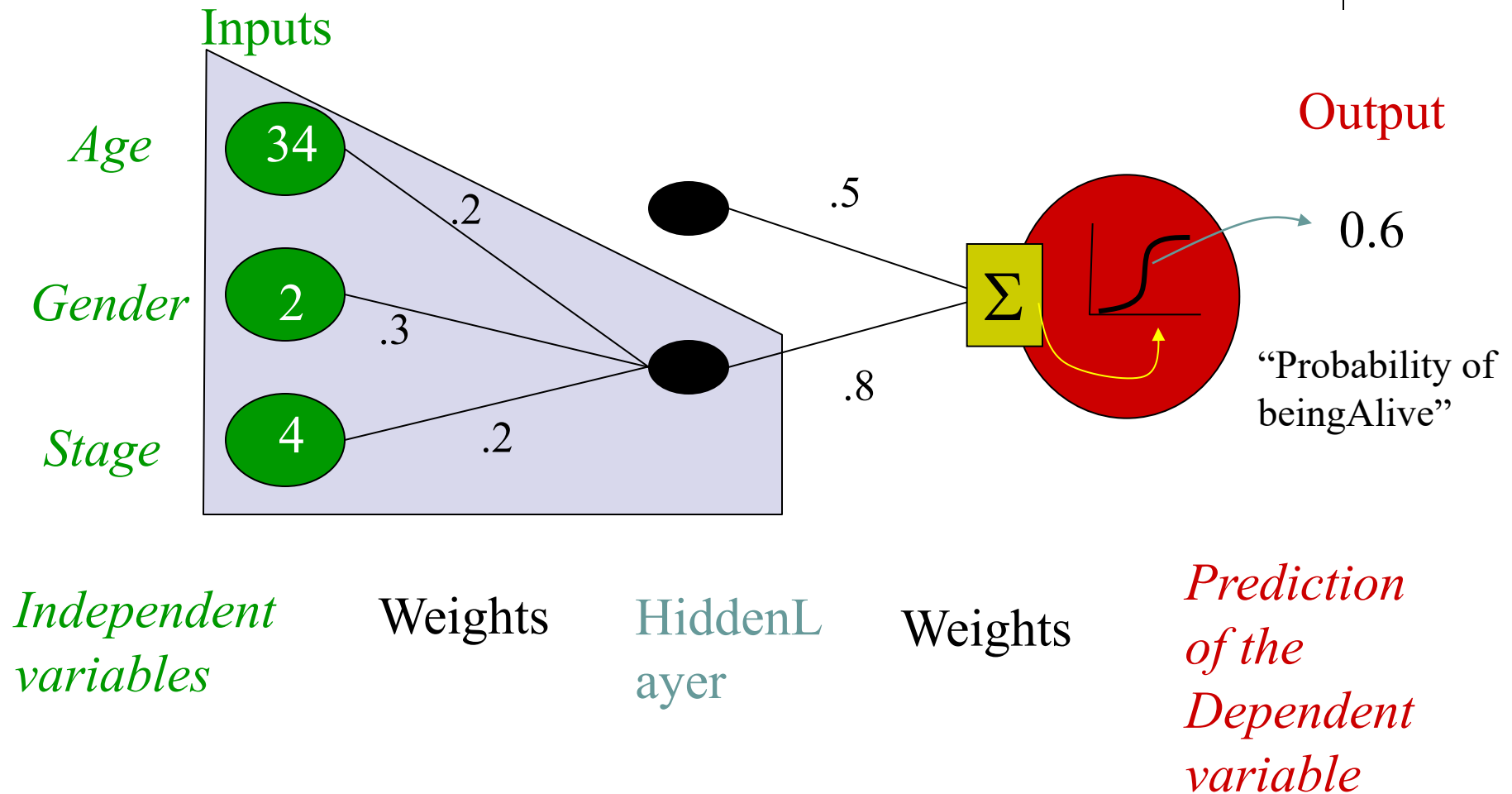
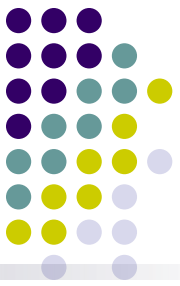
Neural Network Model (with one hidden layer)

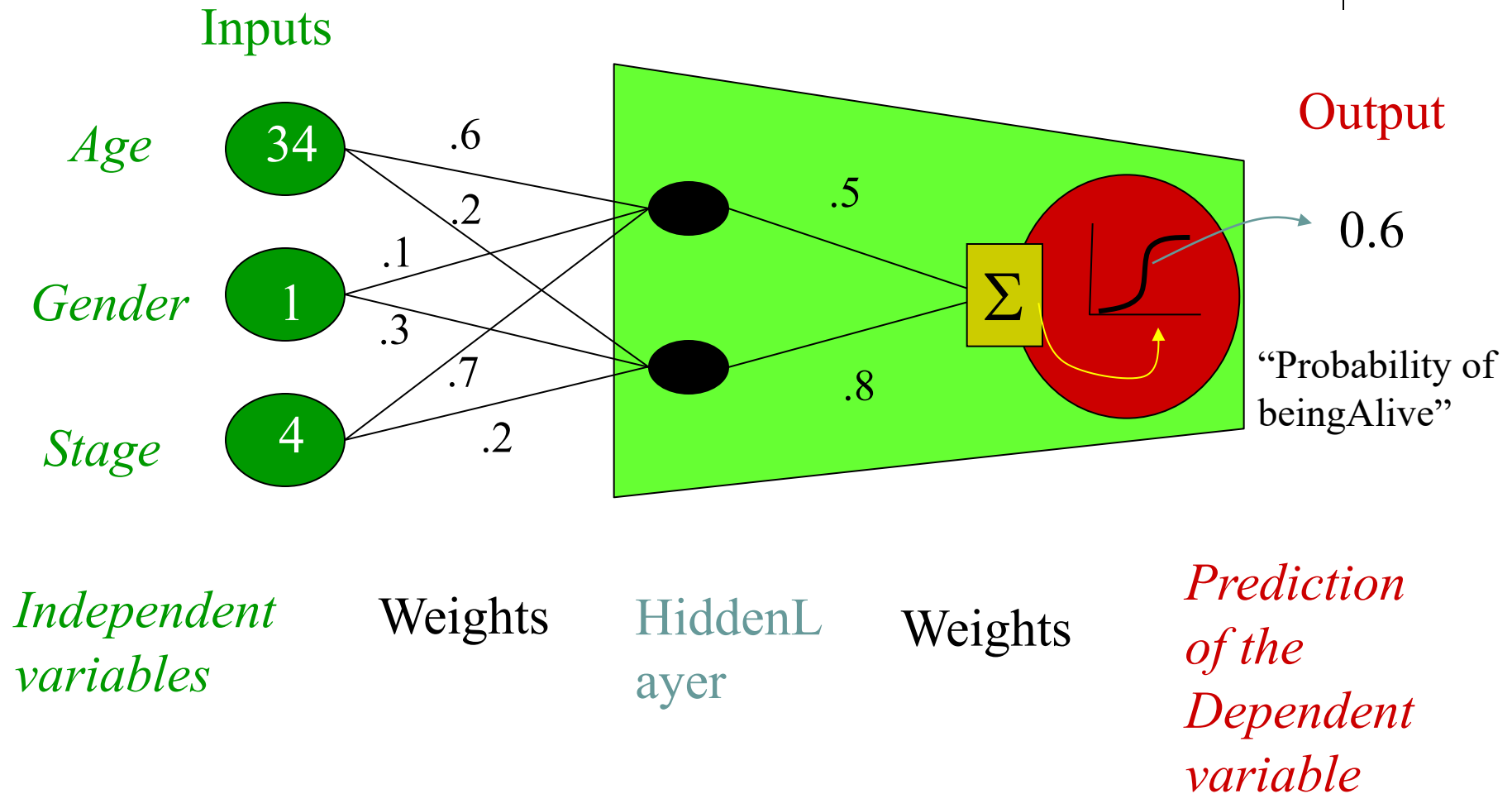
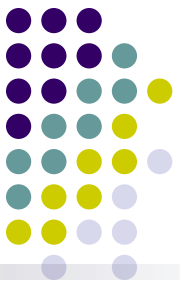


Note: the intercept term (the “bias”) was omitted here for simplicity of the graphic depiction – just add it by yourself, mentally – here and elsewhere in this presentation.

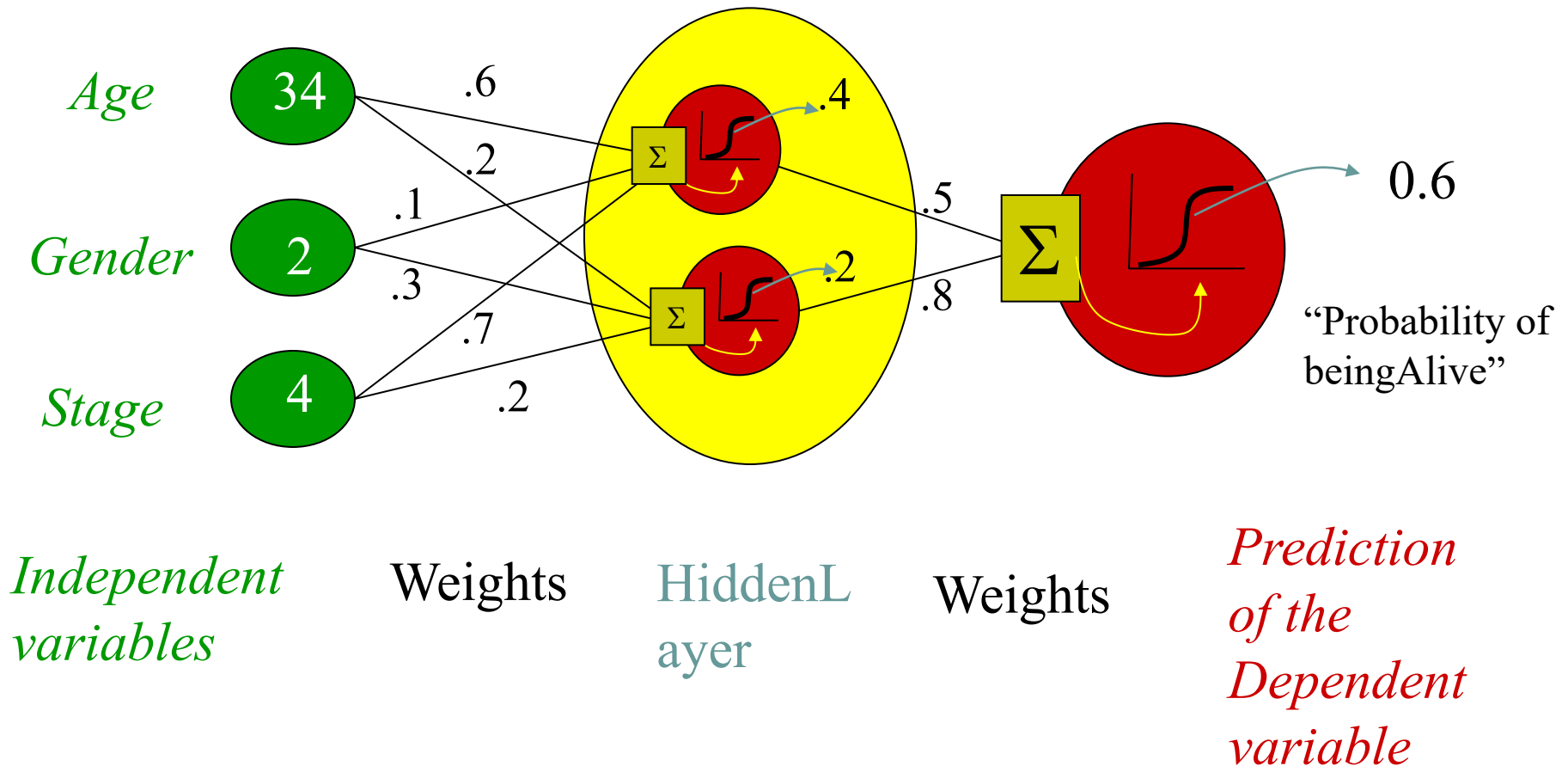
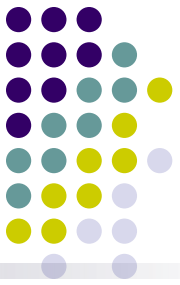
“Combined logistic models”



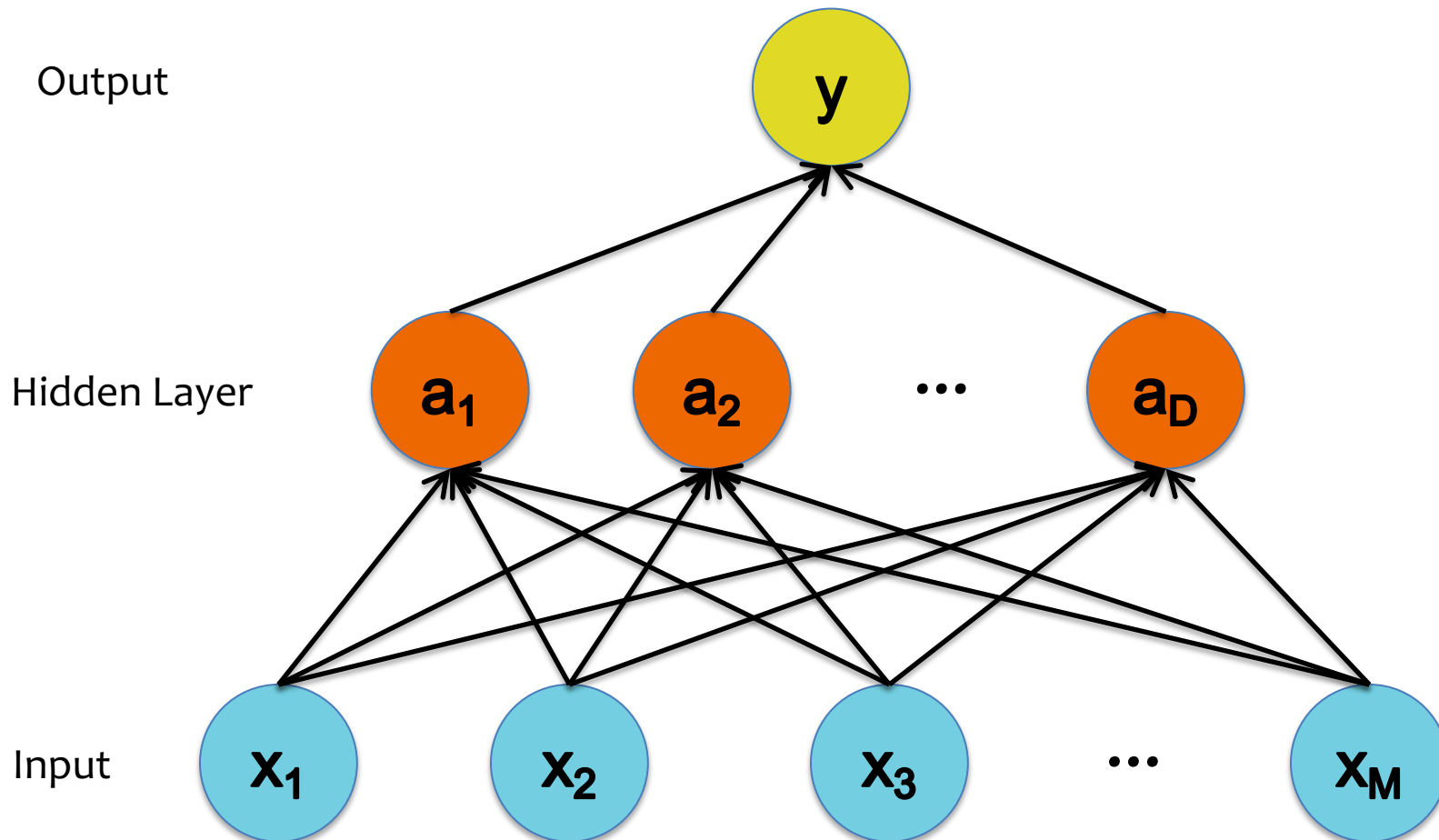




Not really, no target for hidden units...



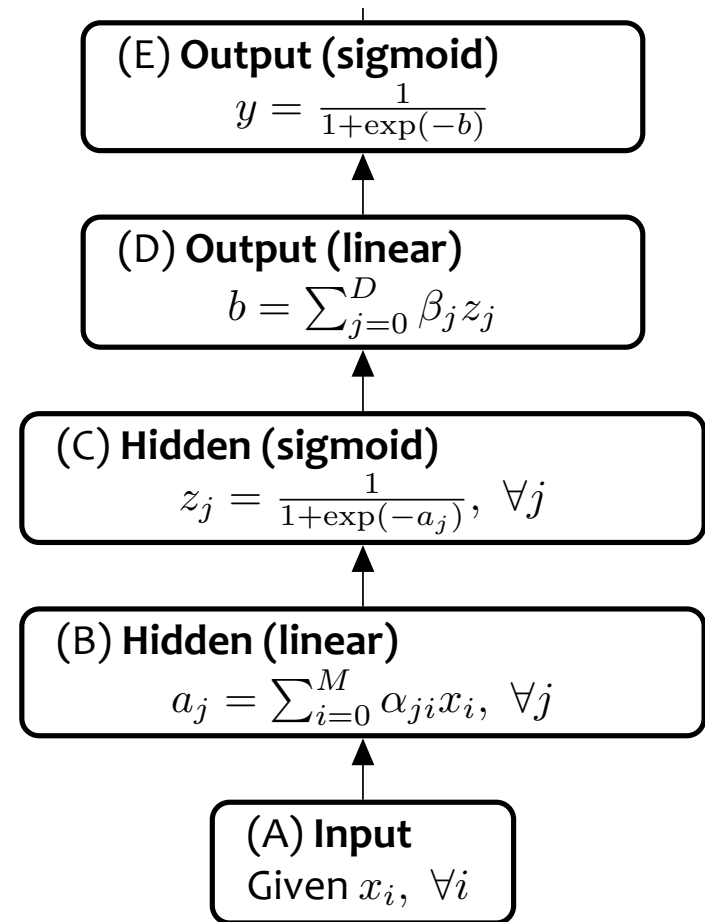
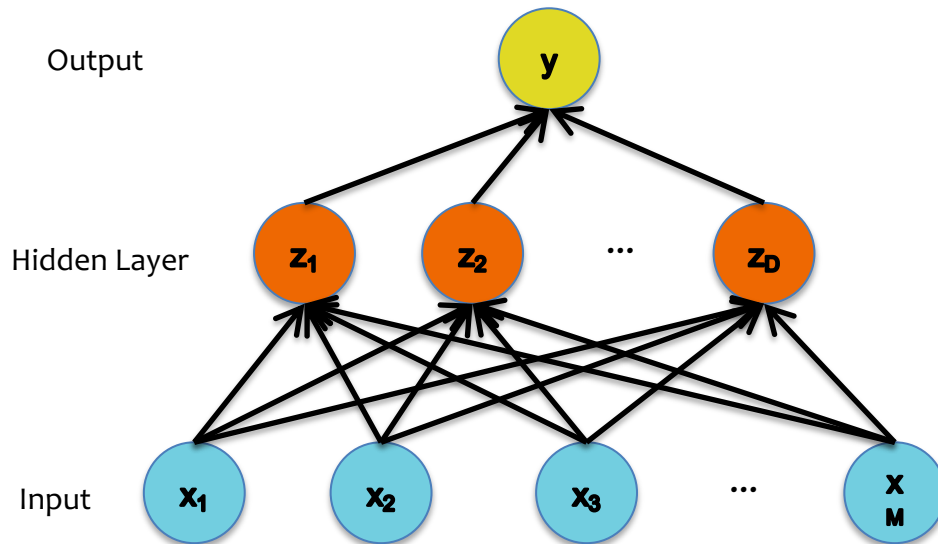
Slide from Matt Gormley, CMU



Decision Functions

Neural Network

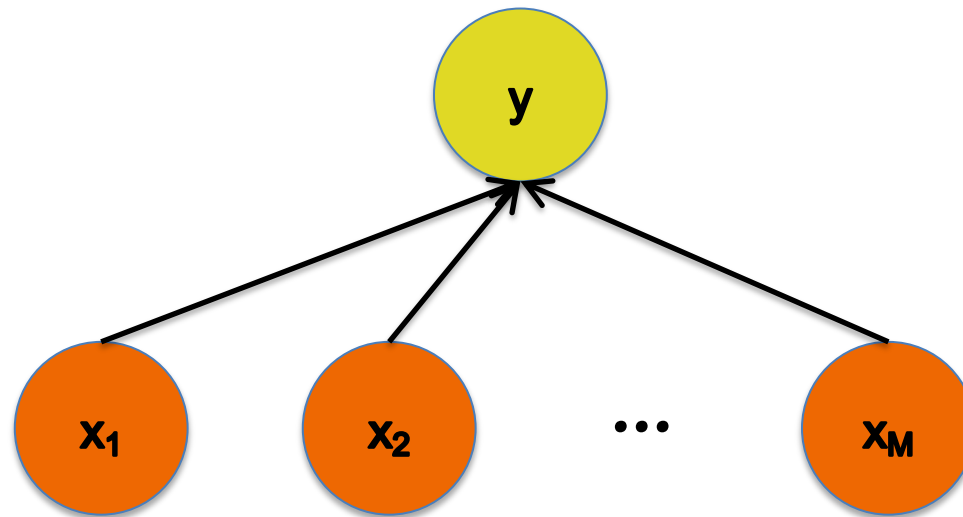
Slide from Matt Gormley, CMU



Building a Neural Net

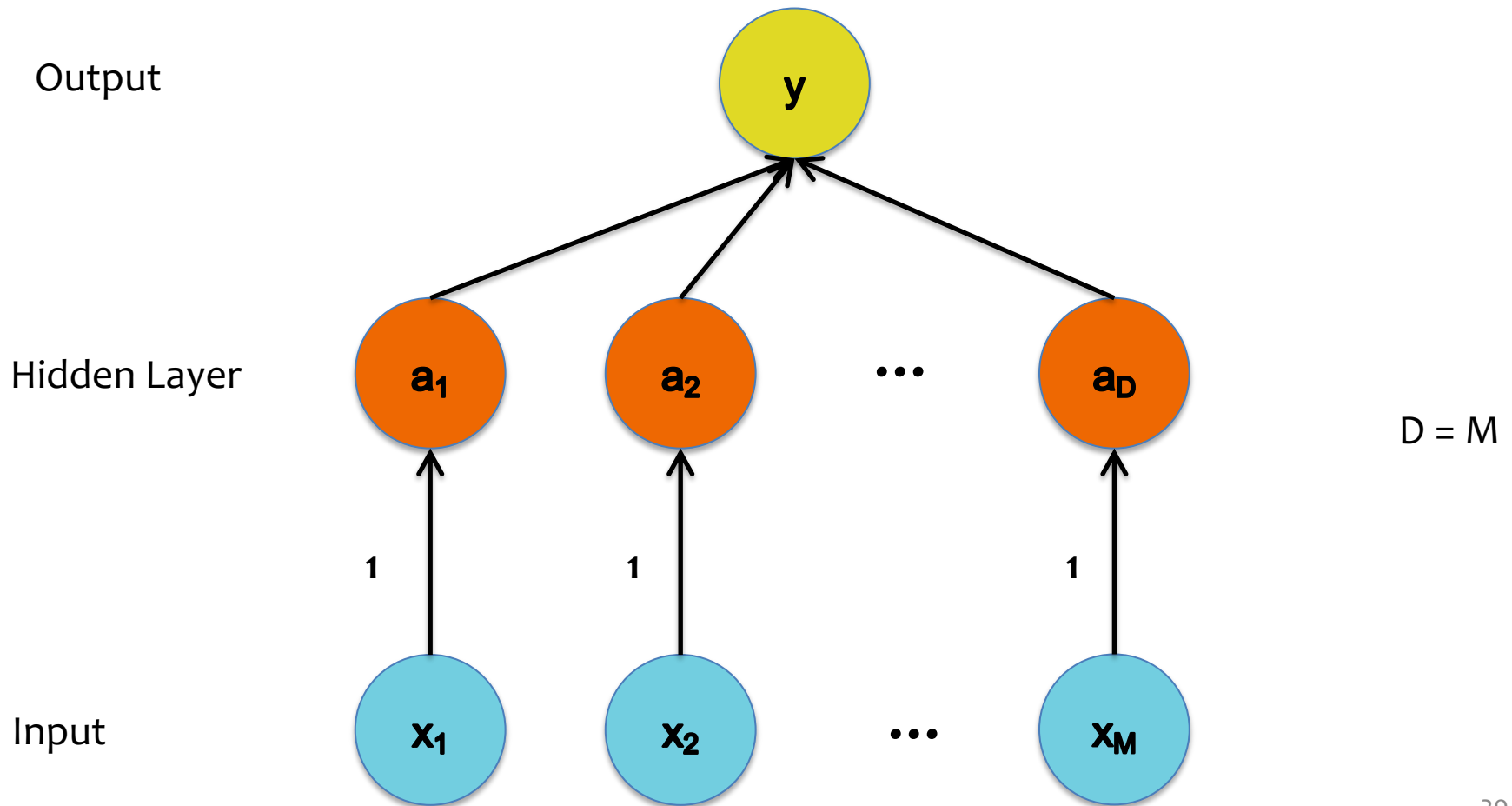
Output

Features



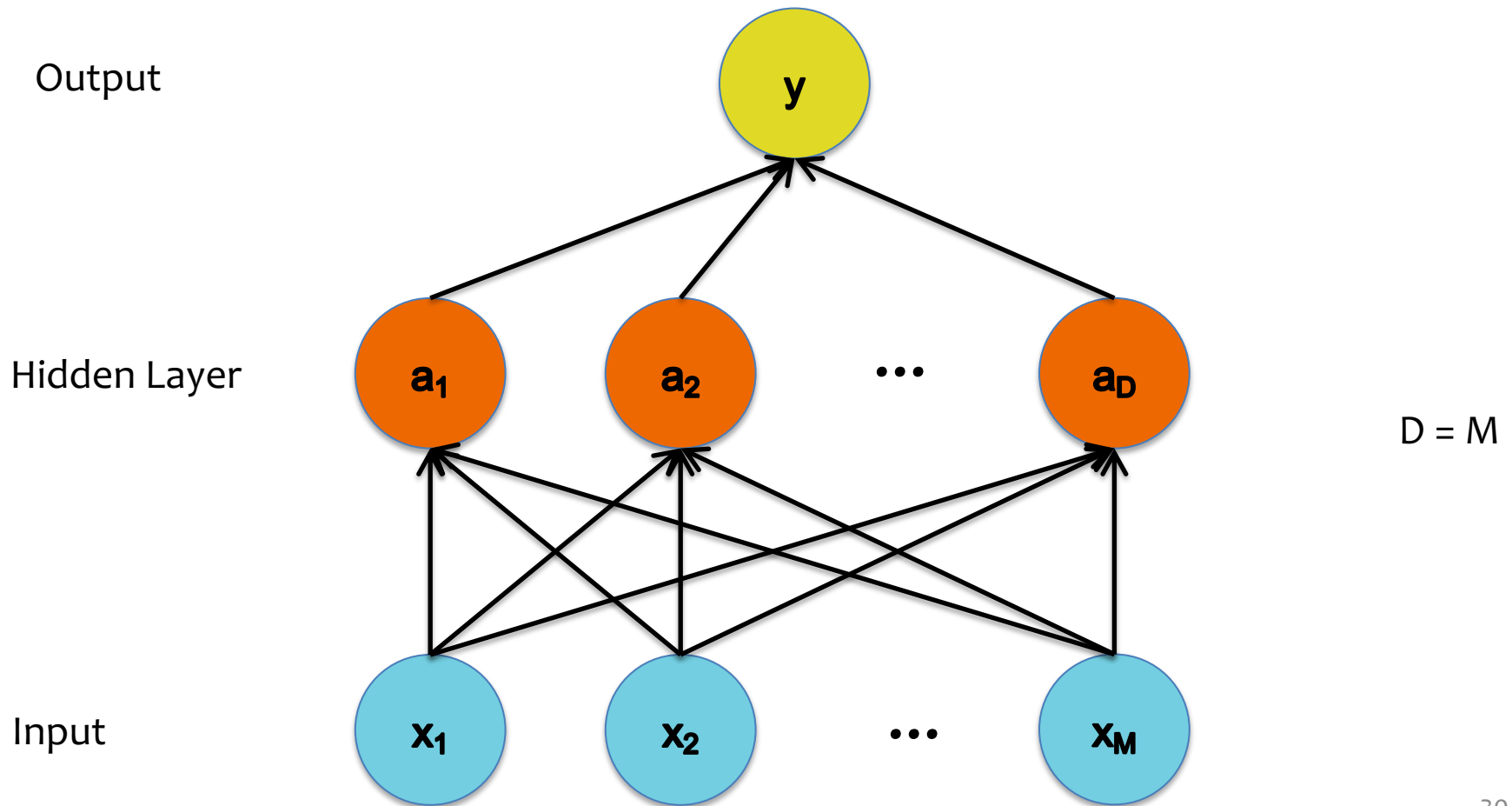
Building a Neural Net

Slide from Matt Gormley, CMU



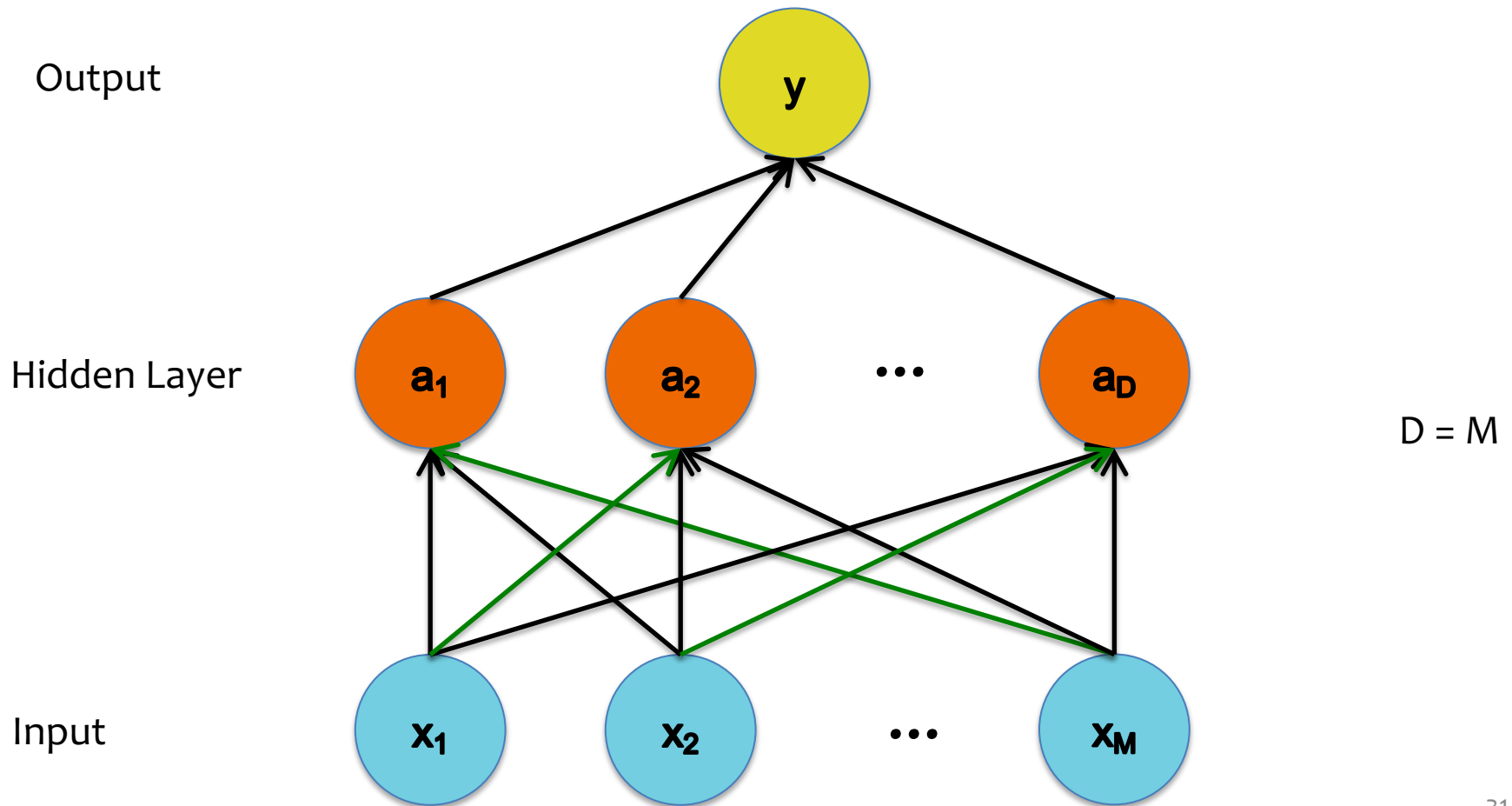
Building a Neural Net

Slide from Matt Gormley, CMU



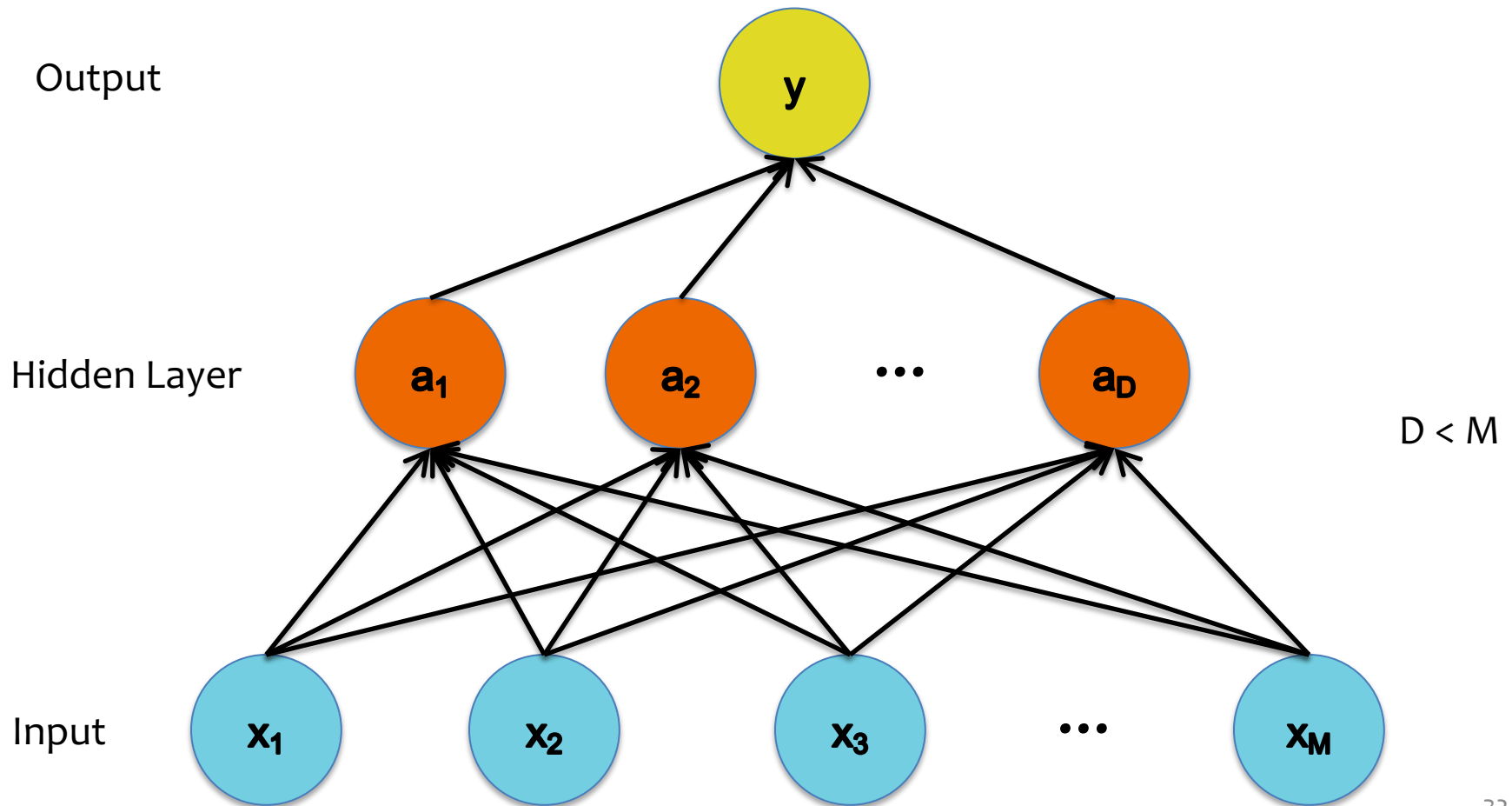
Building a Neural Net

Slide from Matt Gormley, CMU



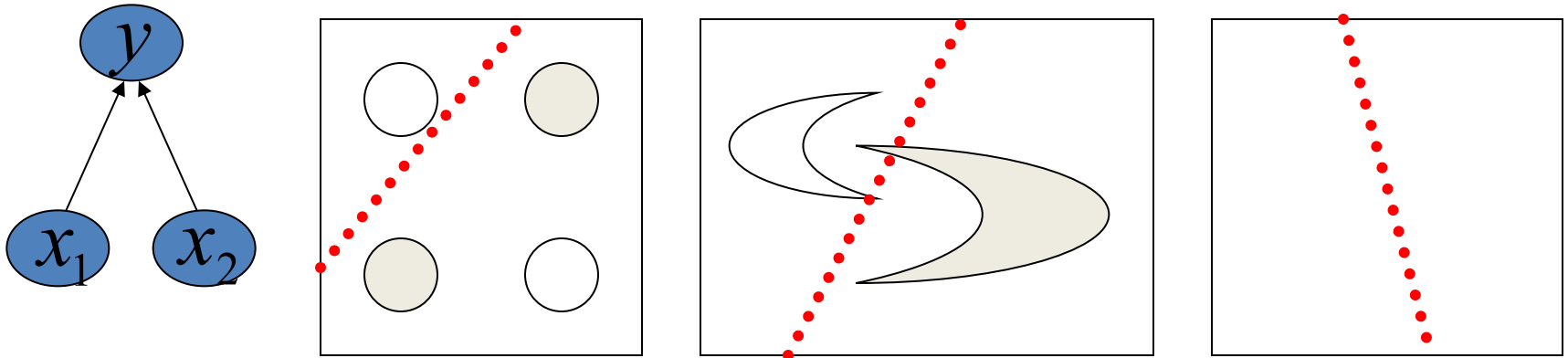
Building a Neural Net

Slide from Matt Gormley, CMU



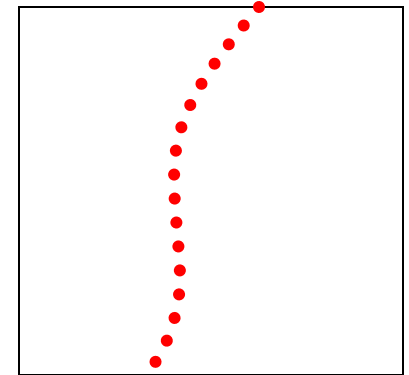
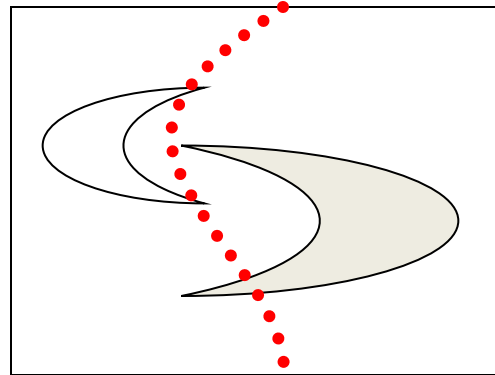
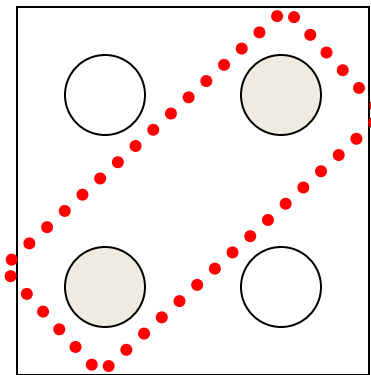
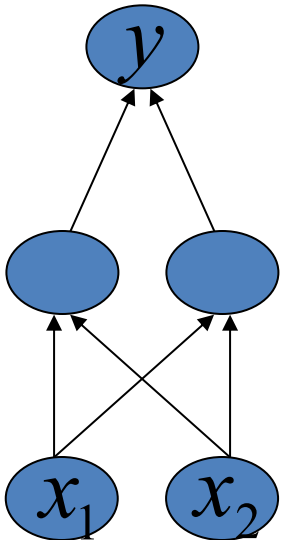
Decision Boundary

- 0 hidden layers: linear classifier
 - Hyperplanes

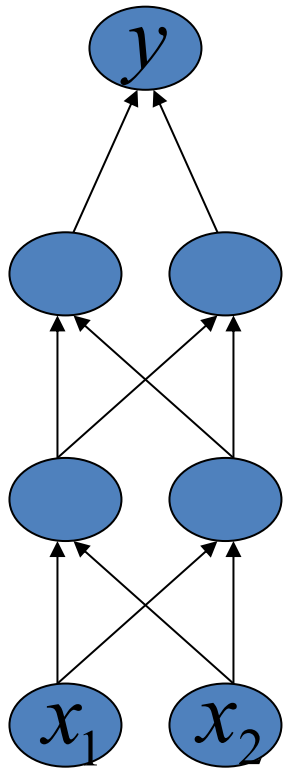


Decision Boundary

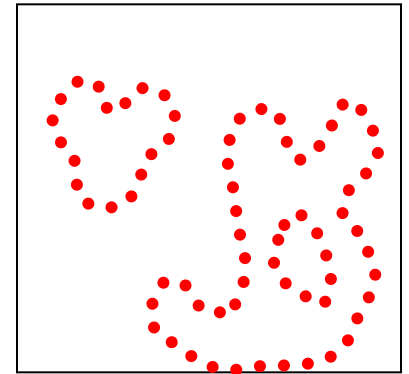
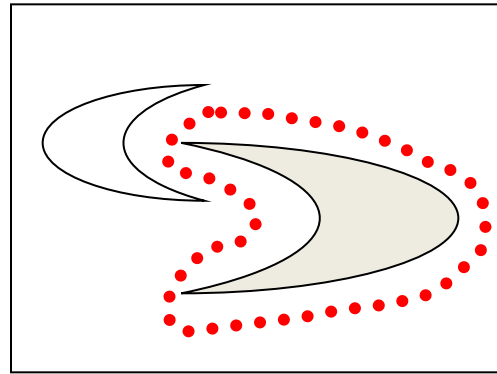
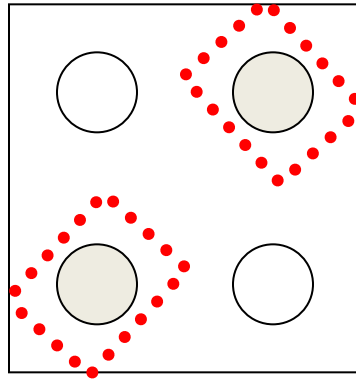
- 1 hidden layer
 - Boundary of convex region (open or closed)



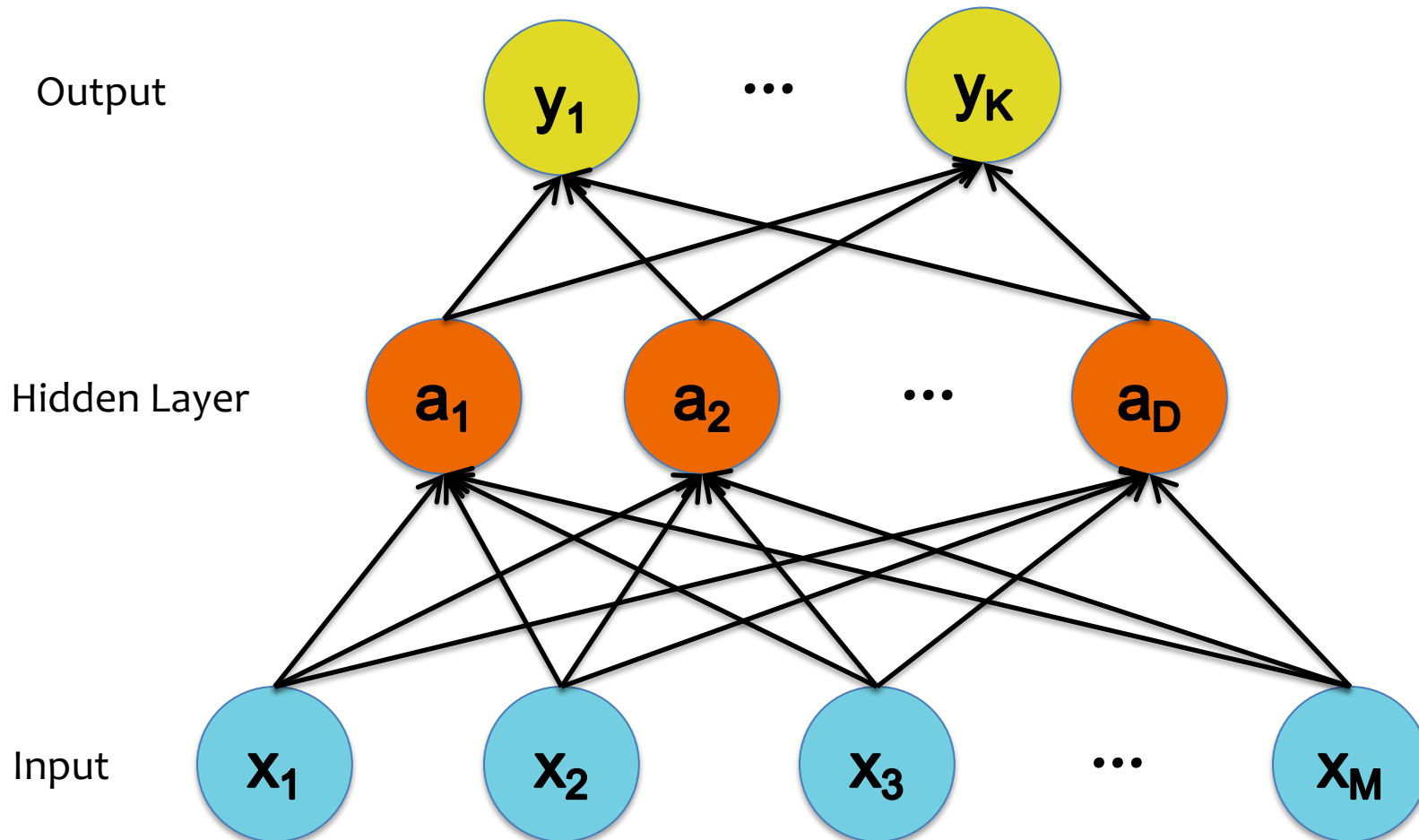
Decision Boundary



- 2 hidden layers
 - Combinations of convex regions



Slide from Matt Gormley, CMU



ARCHITECTURES

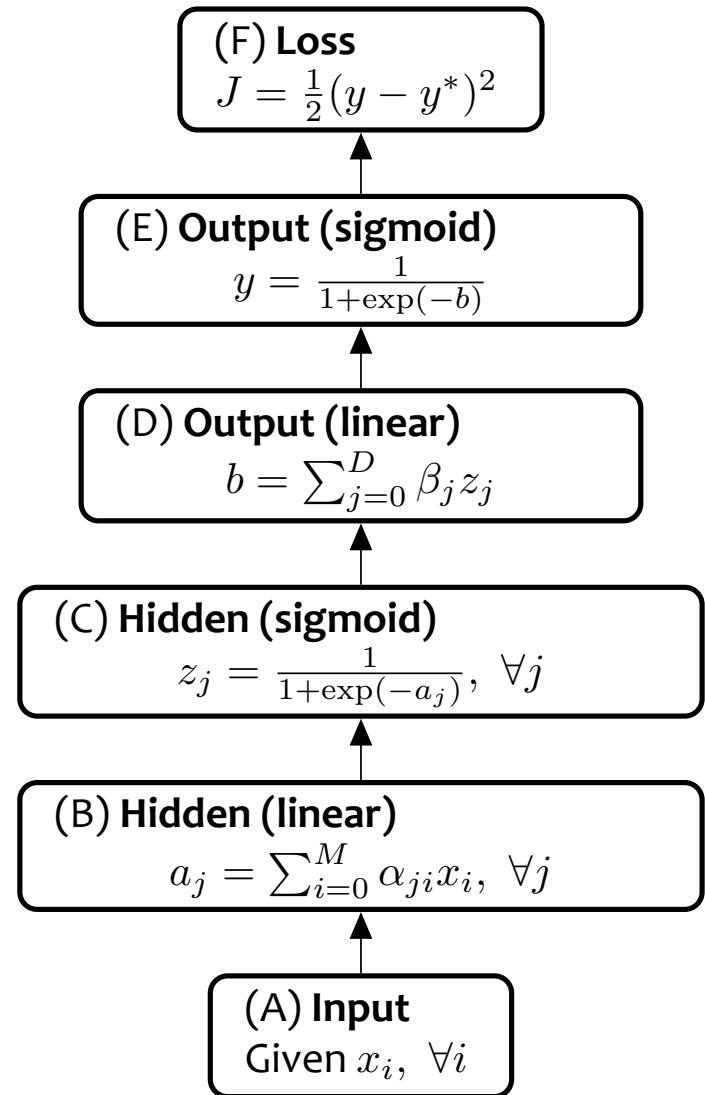
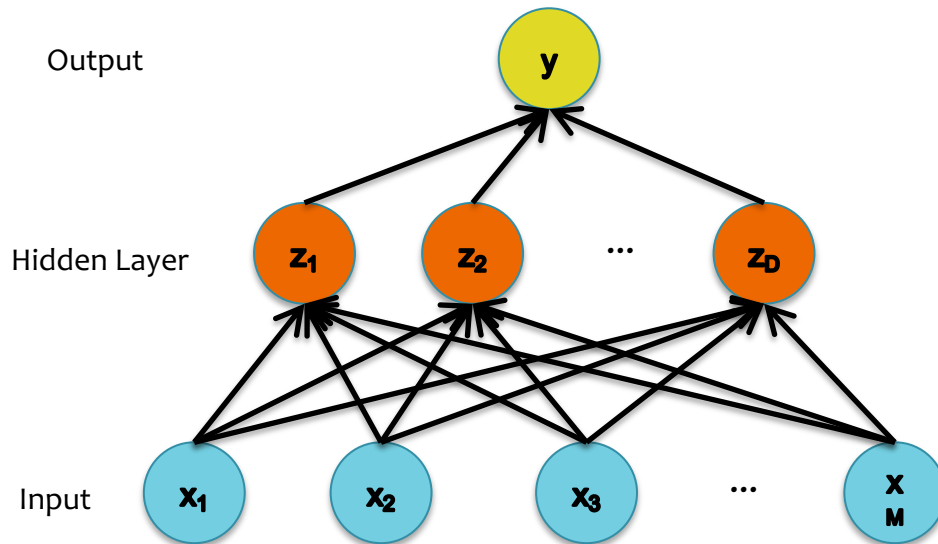
Neural Network Architectures

Even for a basic Neural Network, there are many design decisions to make:

1. # of hidden layers (depth)
2. # of units per hidden layer (width)
3. Type of activation function (nonlinearity)
4. Form of objective function

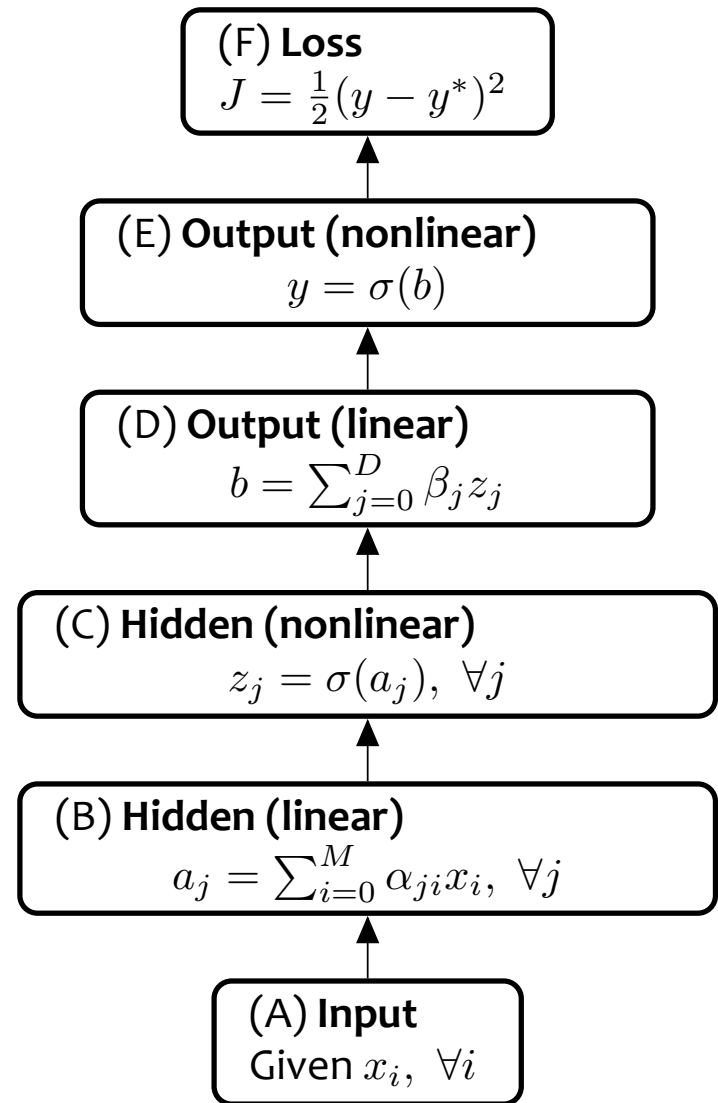
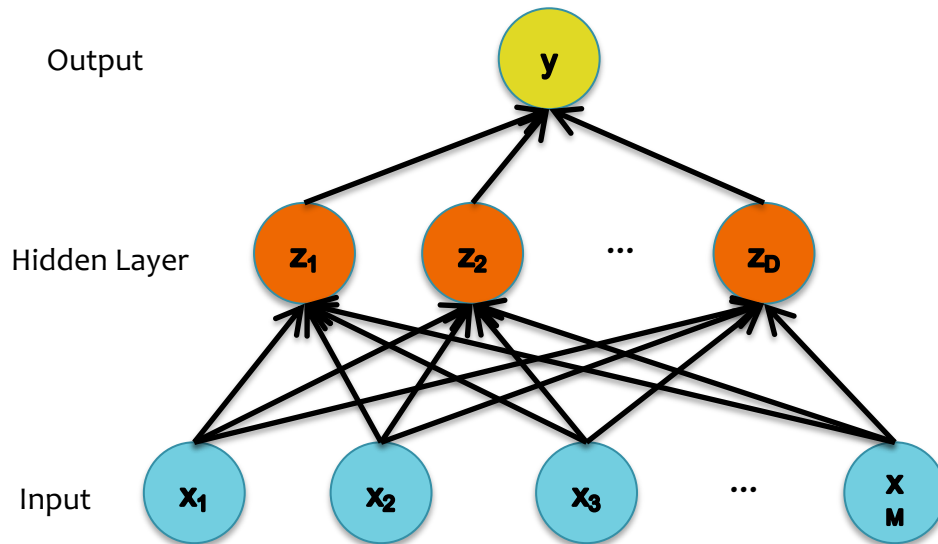
Activation Functions

Neural Network with sigmoid
activation functions



Activation Functions

Neural Network with arbitrary nonlinear activation functions



Activation Functions

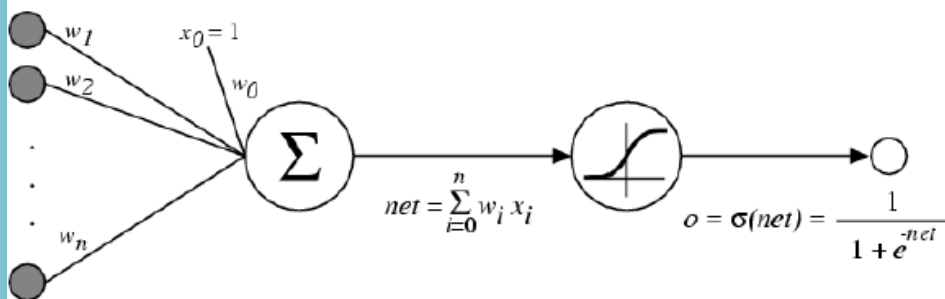
Sigmoid / Logistic Function

$$\text{logistic}(u) \equiv \frac{1}{1 + e^{-u}}$$



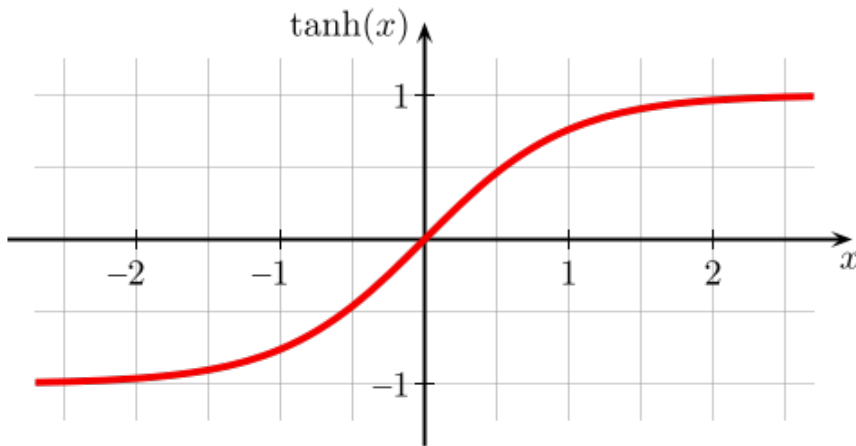
Slide from Matt Gormley, CMU

So far, we've assumed that the activation function (nonlinearity) is always the sigmoid function...



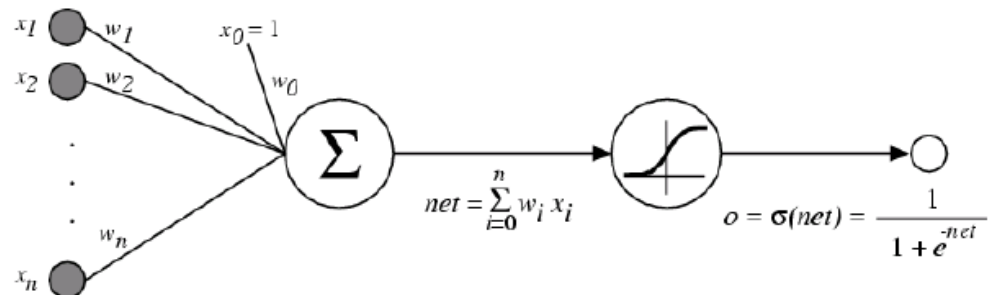
Activation Functions

- A new change: modifying the nonlinearity
 - The logistic is not widely used in modern ANNs



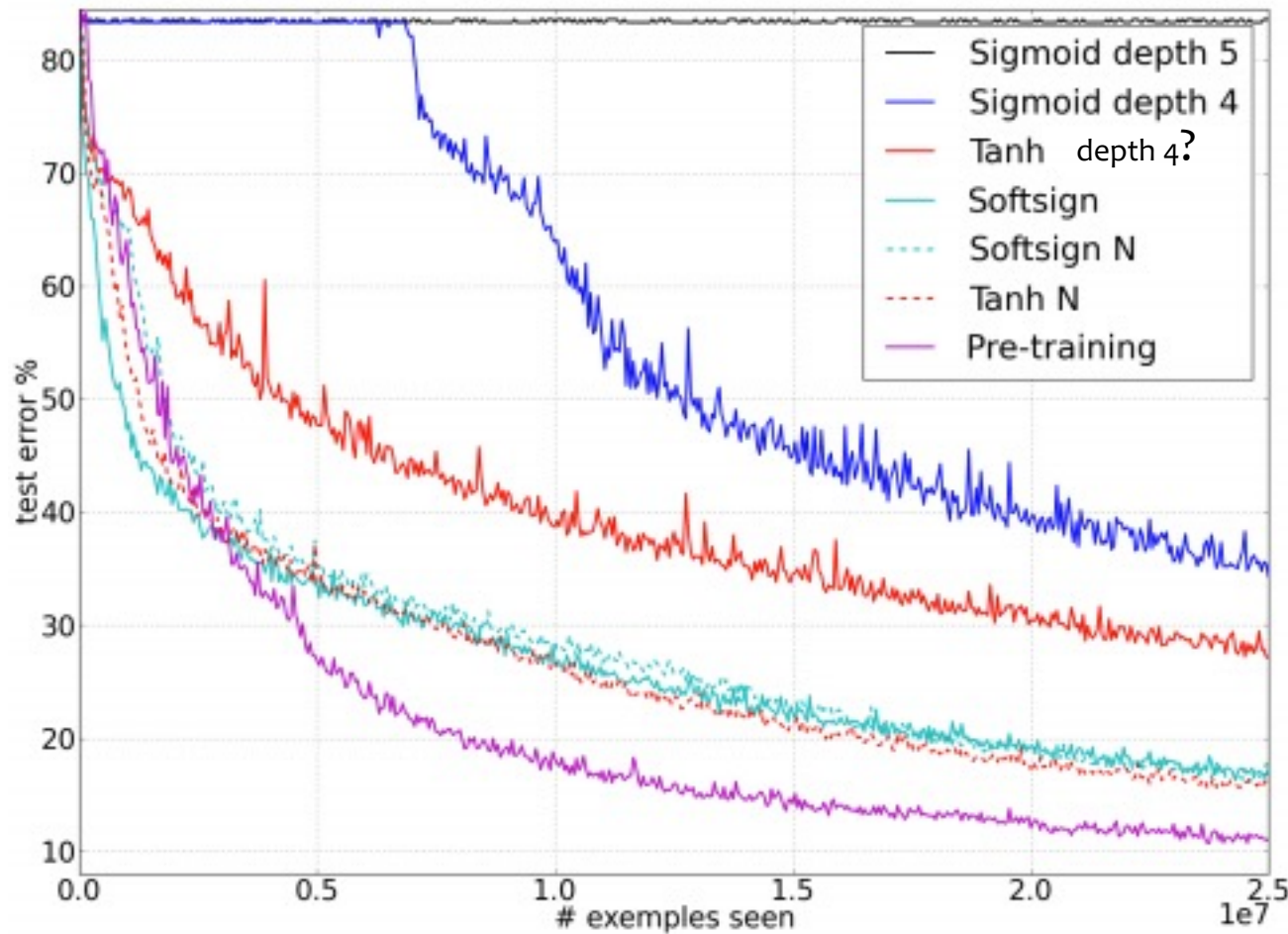
Alternate 1:
tanh

Like logistic function but
shifted to range $[-1, +1]$



Understanding the difficulty of training deep feedforward neural networks

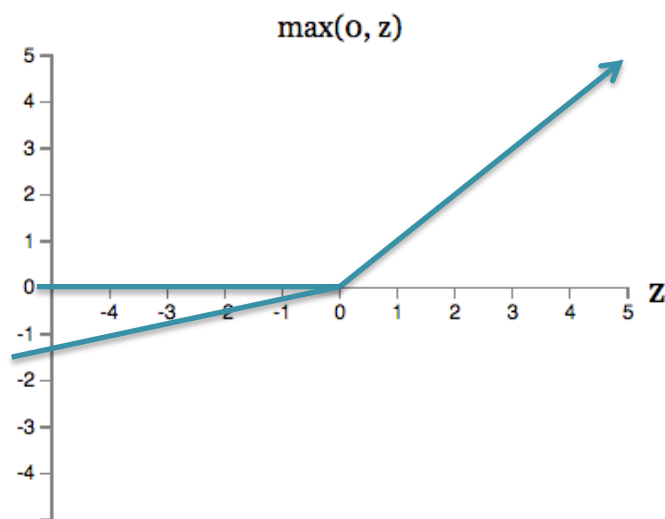
AI Stats 2010



} sigmoid
vs.
tanh

Activation Functions

- A new change: modifying the nonlinearity
 - reLU often used in vision tasks

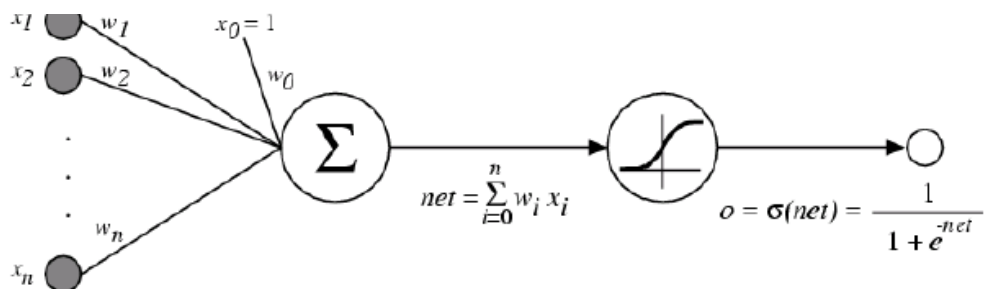


$$\max(0, w \cdot x + b).$$

Alternate 2: rectified linear unit

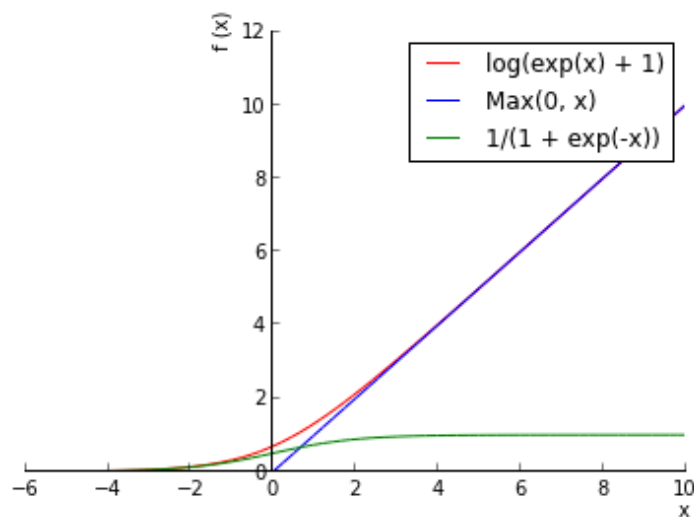
Linear with a cutoff at zero

(Implementation: clip the gradient when you pass zero)



Activation Functions

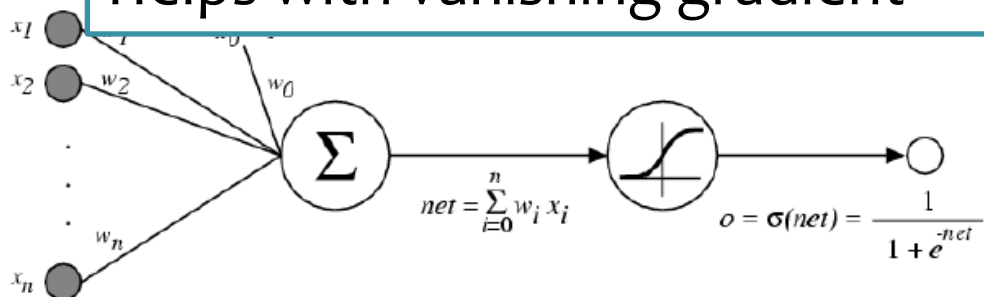
- A new change: modifying the nonlinearity
 - reLU often used in vision tasks



Alternate 2: rectified linear unit

Soft version: $\log(\exp(x)+1)$

Doesn't saturate (at one end)
Sparsifies outputs
Helps with vanishing gradient

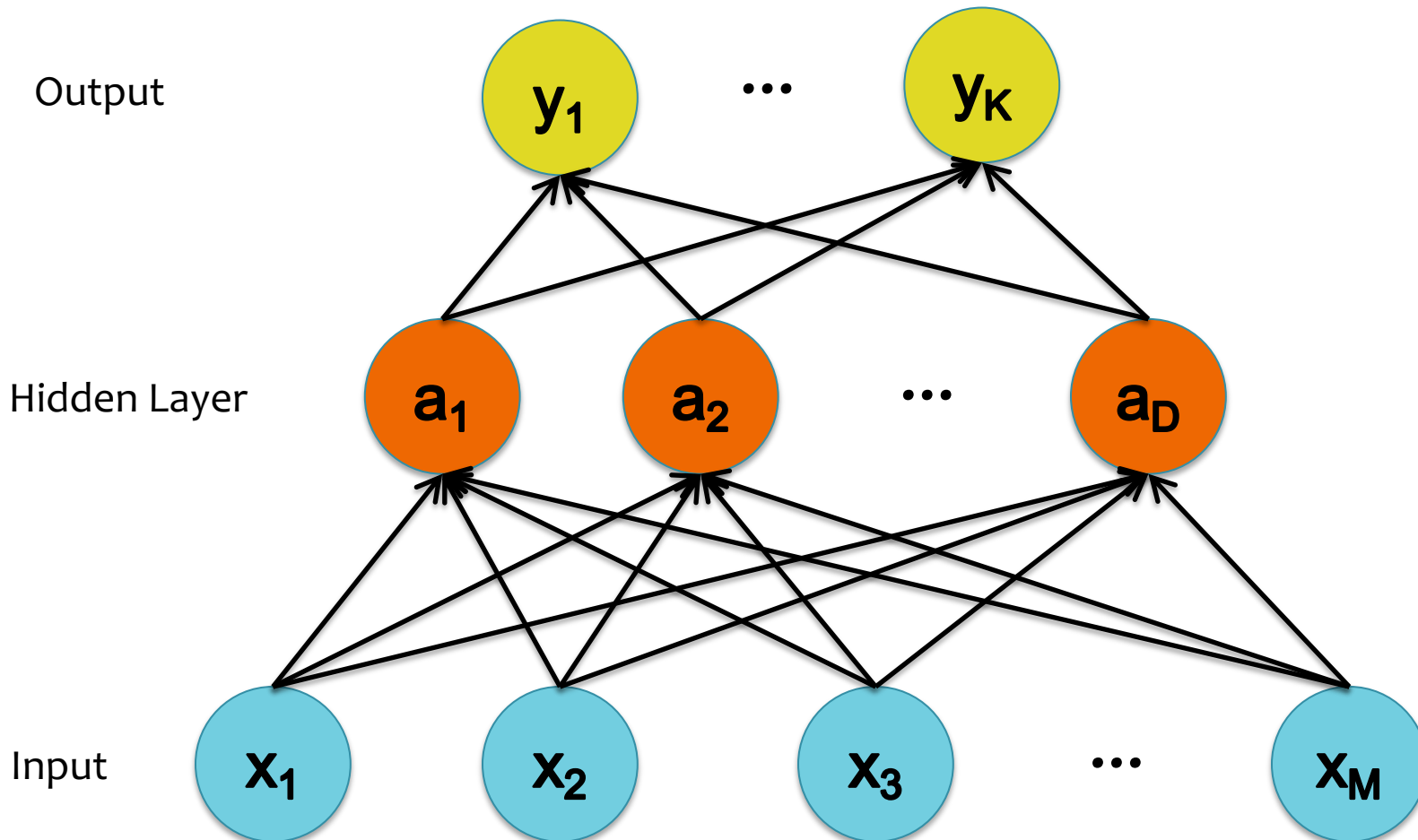


Objective Functions for NNs

- Regression:
 - Use the same objective as Linear Regression
 - **Quadratic loss (or equivalently: SSE)**
- Classification:
 - Use the same objective as Logistic Regression
 - **Cross-entropy (i.e. negative log likelihood)**
 - This requires probabilities, so we add an additional “softmax” layer at the end of our network

Multi-Class Output

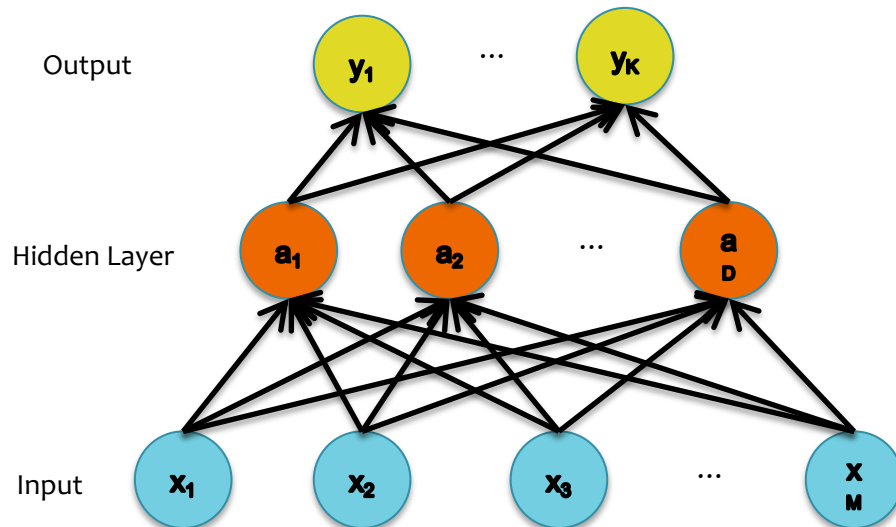
Slide from Matt Gormley, CMU



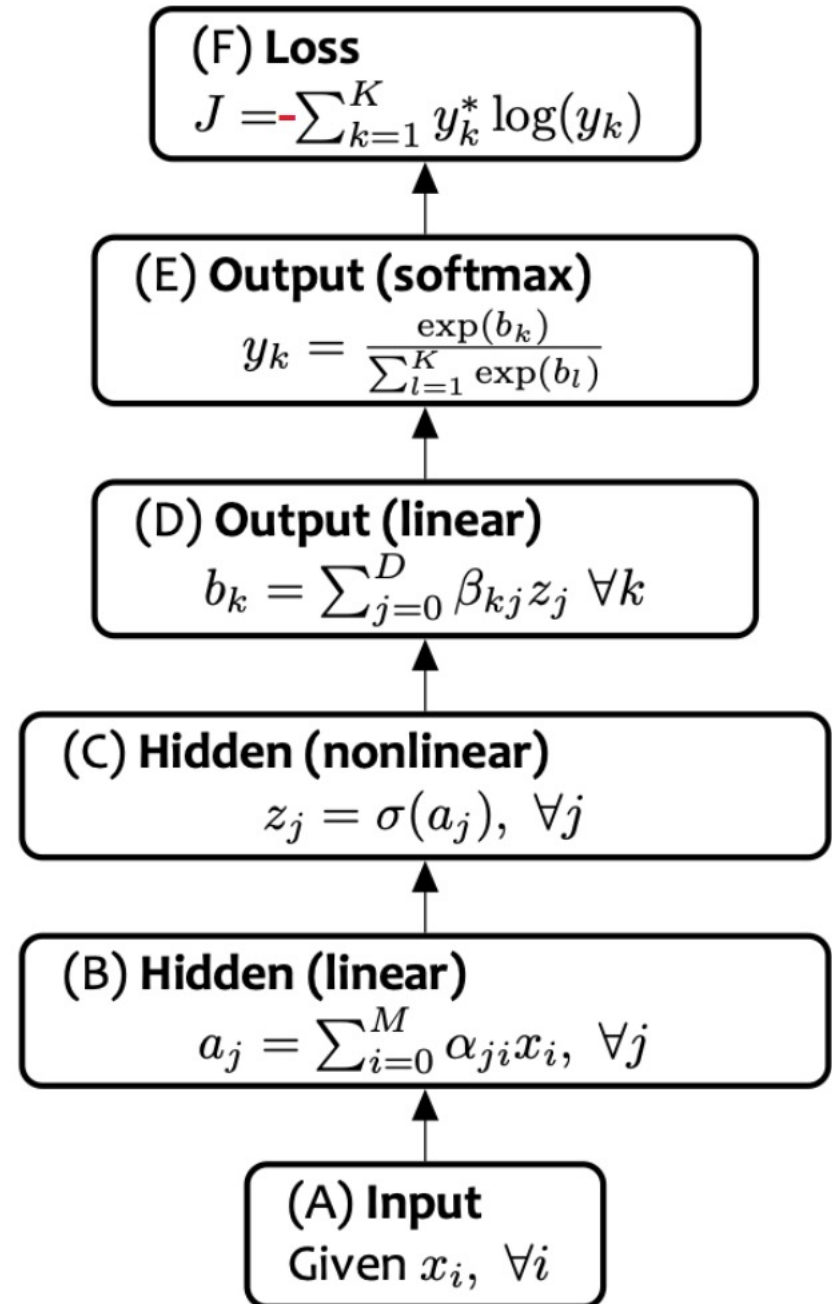
Multi-Class

Softmax:

$$y_k = \frac{\exp(b_k)}{\sum_{l=1}^K \exp(b_l)}$$



Slide from Matt Gormley, CMU



Cross-entropy vs. Quadratic loss

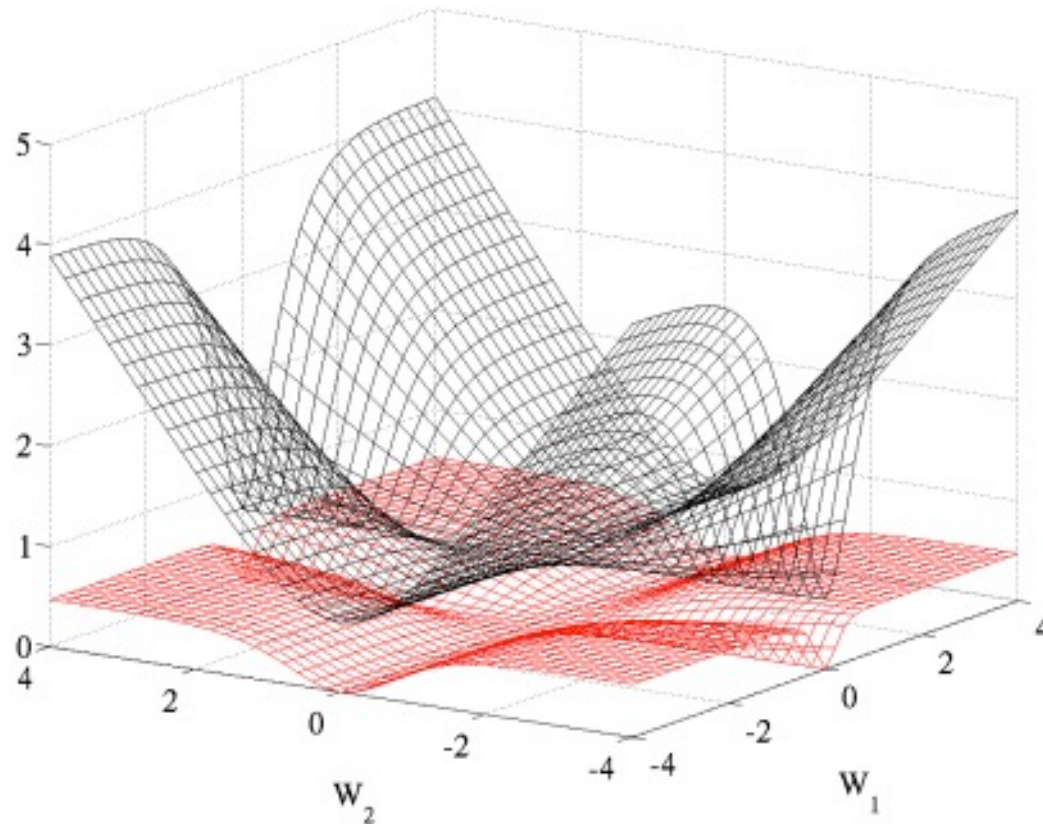


Figure 5: Cross entropy (black, surface on top) and quadratic (red, bottom surface) cost as a function of two weights (one at each layer) of a network with two layers, W_1 respectively on the first layer and W_2 on the second, output layer.

Background

A Recipe for Machine Learning

1. Given training data:

$$\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^N$$

2. Choose each of these:

- Decision function

$$\hat{\mathbf{y}} = f_{\boldsymbol{\theta}}(\mathbf{x}_i)$$

- Loss function

$$\ell(\hat{\mathbf{y}}, \mathbf{y}_i) \in \mathbb{R}$$

3. Define goal:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \sum_{i=1}^N \ell(f_{\boldsymbol{\theta}}(\mathbf{x}_i), \mathbf{y}_i)$$

4. Train with SGD:

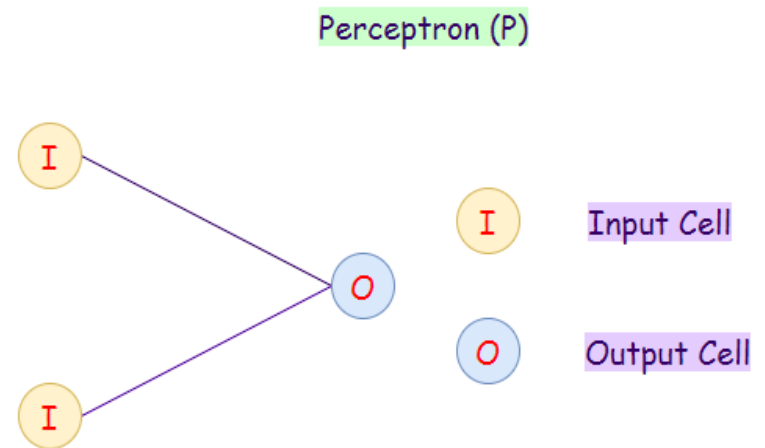
(take small steps
opposite the gradient)

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta_t \nabla \ell(f_{\boldsymbol{\theta}}(\mathbf{x}_i), \mathbf{y}_i)$$

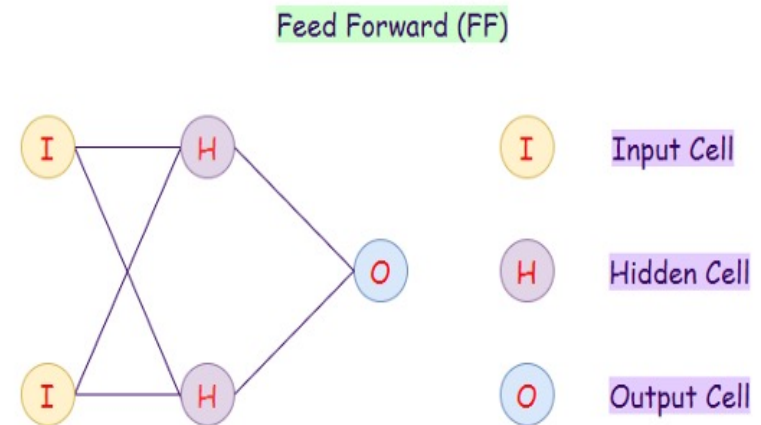
TYPES OF NEURAL NETWORKS

This summary is from my student Jiayou Chao

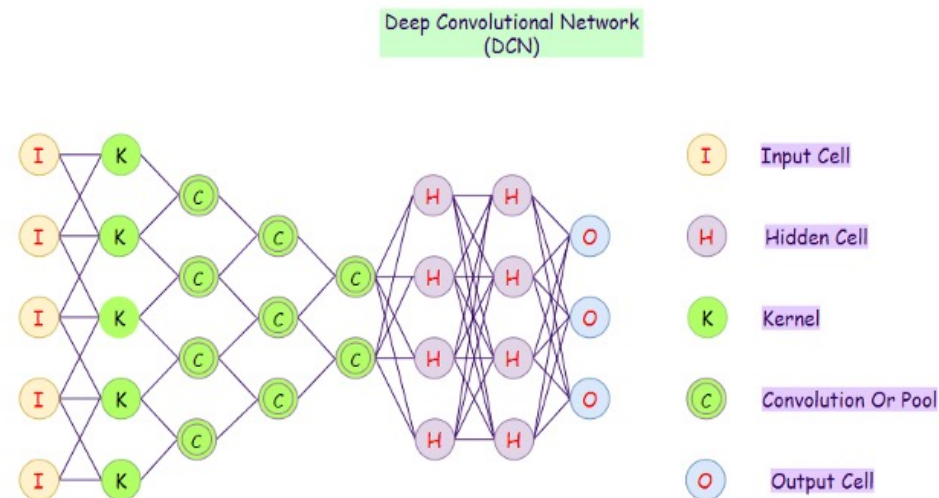
- **Perceptron**
- Feed Forward (Fully-connected) NN
‘Feed forward’ means the nodes do not form a cycle. Therefore, all the nodes are ‘fully connected’, which means all the inputs have a weight to the nodes.
- Convolutional NN
CNN are primarily used for clustering of images and object recognition.
- Markov Chain
Markov Chain is a state machine in which the probability of the next stage is based only on the current stage.
- Recurrent NN
Unlike fully-connected NN, the inputs of RNN have shared weights thus RNN has spatial invariance property.



- Perceptron
- **Feed Forward (Fully-connected) NN**
'Feed forward' means the nodes do not form a cycle. Therefore, all the nodes are 'fully connected', which means all the inputs have a weight to the nodes.
- Convolutional NN
CNN are primarily used for clustering of images and object recognition.
- Markov Chain
Markov Chain is a state machine in which the probability of the next stage is based only on the current stage.
- Recurrent NN
Unlike fully-connected NN, the inputs of RNN have shared weights thus RNN has spatial invariance property.

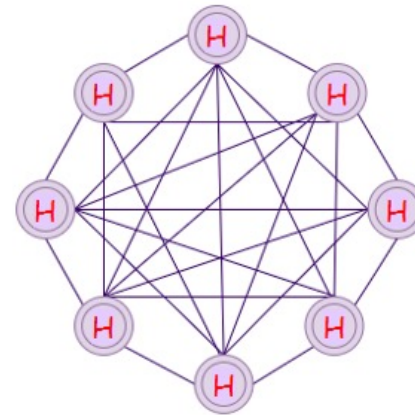


- Perceptron
- Feed Forward (Fully-connected) NN
‘Feed forward’ means the nodes do not form a cycle. Therefore, all the nodes are ‘fully connected’, which means all the inputs have a weight to the nodes.
- **Convolutional NN**
CNN are primarily used for clustering of images and object recognition.
- Markov Chain
Markov Chain is a state machine in which the probability of the next stage is based only on the current stage.
- Recurrent NN
Unlike fully-connected NN, the inputs of RNN have shared weights thus RNN has spatial invariance property.



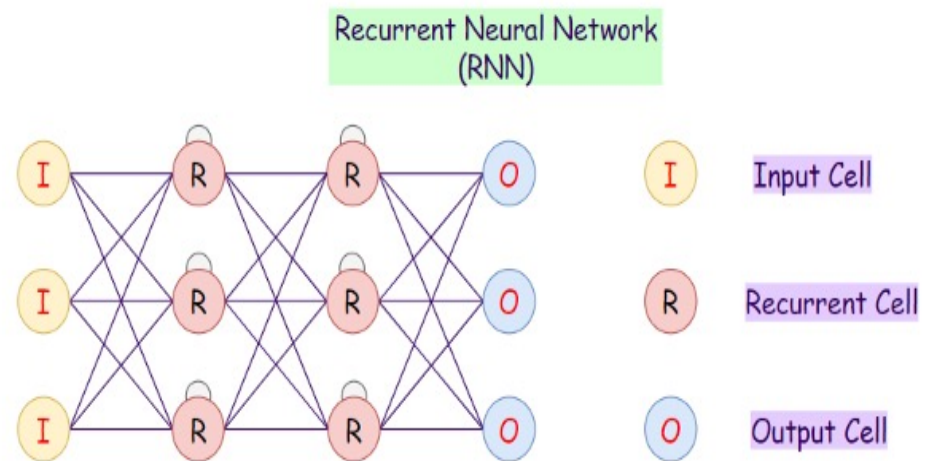
- Perceptron
- Feed Forward (Fully-connected) NN
'Feed forward' means the nodes do not form a cycle. Therefore, all the nodes are 'fully connected', which means all the inputs have a weight to the nodes.
- Convolutional NN
CNN are primarily used for clustering of images and object recognition.
- **Markov Chain**
Markov Chain is a state machine in which the probability of the next stage based only on the current stage.
- Recurrent NN
Unlike fully-connected NN, the inputs of RNN have shared weights thus RNN has spatial invariance property.

Markov Chain (MC)



Probabilistic
Hidden Cell

- Perceptron
- Feed Forward (Fully-connected) NN
‘Feed forward’ means the nodes do not form a cycle. Therefore, all the nodes are ‘fully connected’, which means all the inputs have a weight to the nodes.
- Convolutional NN
CNN are primarily used for clustering of images and object recognition.
- Markov Chain
Markov Chain is a state machine in which the probability of the next stage is based only on the current stage.
- **Recurrent NN**
Unlike fully-connected NN, the inputs of RNN have shared weights thus RNN has spatial invariance property.



NEURAL NETWORK WITH R, PYTHON ETC.

This summary is from my student Jiayou Chao

CRAN package	Supported taxonomy of neural network	Underlying language/vendor
MXNet	Feed-forward, CNN	C/C++/CUDA
darch	RBM, DBN	C/C++
deepnet	Feed-forward, RBM, DBN, autoencoders	R
h2o	Feed-forward network, autoencoders	Java R
nnnet and neuralnet	Feed-forward	R
Keras	Variety of DNNs	Python/keras.io R
TensorFlow	Variety of DNNs	C++, Python/Google R

Please refer to the following website for a more detailed classification of neural networks:

<https://towardsdatascience.com/the-mostly-complete-chart-of-neural-networks-explained-3fb6f2367464>

**NOW LET'S
DO IT
OURSELVES!**



We Can Do It. Color poster by J. Howard Miller. 179-WP- 1563.*